POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Novel attack strategies targeting PROFIbus and PROFIsafe Exploiting Error Management vulnerabilities

Tesi di Laurea Magistrale in
Computer Science and Engineering - Ingegneria Informatica

Author: **Alessio Alfonsi**

Student ID: 991078
Advisor: Stefano Longari
Academic Year: 2023-24

Abstract

The increasing integration of digital technologies and network connectivity in industrial environments has significantly expanded the attack surface of Industrial Control Systems (ICSs). The communication protocols supporting these systems, particularly at lower levels, often lack robust authentication and encryption mechanisms, making them vulnerable to exploitation. This research investigates and exploits vulnerabilities related to Error Management within industrial communication protocols, specifically at the data-link level (UART), within PROFIbus and PROFIsafe environments. The analysis is supported by the development of practical methods to exploit these vulnerabilities and assess their impact on the system. The experimental results challenge the assumption (considered secure) that a channel can only be controlled by a designated Controller. We definitively demonstrate that this assumption is flawed and that it is possible to gain control of the entire network by compromising any device within it. This includes contributions such as a new Reparameterization Attack that targets networks using the PROFIbus protocol and an enhancement to an existing attack on the PROFIsafe protocol, demonstrating how to efficiently manipulate the CRC2 checksum to bypass safety mechanisms. Finally, we propose concrete countermeasures to address the identified vulnerabilities at different layers of the protocol stack, with the intention of improving the security of critical systems against cyber threats.

Keywords: Industrial Control Systems, UART, PROFIbus, PROFIsafe, Cybersecurity, Error Management, Data-Link Layer, Attacks, Countemeasures, Reparameterization, CRC Manipulation

Abstract in lingua italiana

L'integrazione di connettività e tecnologie avanzate negli ambienti industriali ha ampliato significativamente la superficie di attacco negli Industrial Control Systems (Sistemi di Controllo Industriale). I protocolli di comunicazione che supportano questi sistemi, in particolare a basso livello, mancano di robusti meccanismi di autenticazione e crittografia, rendendoli vulnerabili. Questo lavoro analizza e sfrutta le vulnerabilità legate alla gestione degli errori nei protocolli di comunicazione industriale, specificamente al livello data-link (UART), in ambienti PROFIbus e PROFIsafe. L'analisi è supportata dallo sviluppo di attacchi reali che sfruttano queste vulnerabilità e permettono di valutarne l'impatto sul sistema. I risultati sperimentali mettono in discussione il concetto (considerato sicuro) secondo cui un canale di comunicazione possa essere controllato solo da un Controller designato. Dimostriamo in modo definitivo che questa assunzione è errata e che è possibile ottenere il controllo dell'intera rete compromettendo un qualsiasi dispositivo al suo interno. Questo include contributi come un nuovo attacco di riparametrizzazione di un Device, che prende di mira le reti che utilizzano il protocollo PROFIbus, e un miglioramento ad un attacco esistente sul protocollo PROFIsafe, manipolando il checksum CRC2 per superare i meccanismi di sicurezza. Infine proponiamo delle contromisure per mitigare le vulnerabilità identificate nei protocolli, con l'intenzione di migliorare la sicurezza dei sistemi critici contro gli attacchi che abbiamo introdotto.

Parole chiave: Sistemi di Controllo Industriale, UART, PROFIbus, PROFIsafe, Sicurezza Informatica, Gestione degli Errori, Livello Data-Link, Attacchi, Countromisure, Riparametrizzazione, Manipolazione della CRC

Contents

Abstract **i**

Abstract in lingua italiana **iii**

Contents **v**

1 Introduction **1**

2 Background **5**

2.1 Industrial Control Systems 5
2.1.1 Layers of ICS 6
2.1.2 PLCs and Industrial Networks 7
2.2 Fieldbus 7
2.3 PROFIbus 8
2.3.1 Key Characteristics 8
2.3.2 Frames 9
2.3.3 Devices 10
2.3.4 Cyclic Communication 10
2.3.5 Error Management 10
2.4 PROFIsafe 11
2.5 UART 12
2.5.1 Key Characteristics of UART 12
2.5.2 Relationship with RS-485 and PROFIbus 13
2.5.3 Frames and Communication 13
2.5.4 Error Management 14
2.6 RS-485 14
2.6.1 Key Characteristics of RS-485: 14
2.6.2 Electrical Functioning 15
2.7 Safety & Security 15

3 Motivation **17**
3.1 Problem Statement . . . 17
3.2 State of the Art . . . 18
3.2.1 Attacks on UART and RS485 . . . 19
3.2.2 PROFIbus and other fieldbus attacks . . . 20
3.2.3 PROFIsafe Attacks . . . 23
3.2.4 Countermeasures . . . 24
3.3 Goals and Challenges . . . 26

4 Threat Model **29**
4.1 Scope and Applicability . . . 29
4.2 Assets and Threat Agents . . . 30
4.2.1 Assets . . . 30
4.2.2 Threat Agents . . . 31
4.2.3 Mapping . . . 32
4.3 Assumptions . . . 33
4.4 Attack Surface . . . 34
4.5 Impact . . . 36

5 Attacks **39**
5.1 Reparameterization Attack (PROFIbus) . . . 39
5.2 PROFIsafe CRC2 Manipulation . . . 47

6 Countermeasures **57**
6.1 Protocol Countermeasures . . . 58
6.2 Software Countermeasures . . . 60

7 Implementation **63**
7.1 Software Implementation . . . 63
7.2 Hardware Implementation . . . 66

8 Experimental Validation **67**
8.1 Reparameterization Attack (PROFIbus) . . . 67

9 Future Works **73**
9.1 Limitations . . . 73
9.2 Future Research . . . 73

10 Conclusions **75**

Bibliography **77**

List of Figures **81**

List of Tables **83**

Acknowledgements **85**

1 | Introduction

The increasing integration of digital technologies and network connectivity within industrial environments has significantly expanded the attack surface and potential for cyber threats to Industrial Control Systems (ICSs). While these technological advancements offer numerous benefits, such as improved efficiency and productivity, they also introduce new vulnerabilities that can be exploited by malicious actors. The communication protocols supporting these systems, particularly at the lower levels, were often developed in an era where security was not a primary concern. Protocols like PROFIbus and PROFIsafe, which are central to this thesis, lack robust authentication and encryption mechanisms, making them vulnerable to exploitation.

The lack of robust security measures in these protocols, coupled with the increasing integration of ICS networks with corporate networks and the Internet, has made them attractive targets for cyberattacks. The potential consequences of such attacks are severe, ranging from disruptions in production and financial losses to safety hazards and even threats to human life.

The Stuxnet malware, which targeted Iranian nuclear enrichment facilities, manipulating specific centrifuges to operate at destructive speeds, rendering them unusable, and resulting in an estimated 20% of Iran's centrifuges being compromised. The BlackEnergy attack, which targeted three Ukrainian power distribution utilities, caused a blackout for several hours, impacting hundreds of thousands of customers. These real-world examples underscore the urgent need for enhanced security measures to protect critical infrastructure.

Several approaches have been proposed to enhance the security of ICS. Intrusion detection systems (IDS) aim to identify and mitigate attacks by monitoring network traffic and system behavior for anomalies. However, the challenge lies in distinguishing between legitimate errors and malicious activities, which can lead to false positives and disrupt normal operations. Encryption has also been explored as a means to protect data confidentiality and integrity. However, its implementation in ICS can be challenging due to the real-time processing constraints and the need to maintain compatibility with legacy sys-

tems. Moreover, encryption alone may not be sufficient to prevent sophisticated attacks that exploit vulnerabilities at lower layers of the protocol stack.

This thesis focuses on investigating and exploiting vulnerabilities related to error management within industrial communication protocols, specifically UART, PROFIbus, and PROFIsafe. By comprehensively understanding the weaknesses inherent in these protocols, we aim to develop novel attack strategies that can compromise the integrity and availability of industrial systems. The focus on the data link layer, where the UART protocol resides, is motivated by its foundational role in communication within many ICSs. The research explores both theoretical and practical vulnerabilities present at this layer, with a particular emphasis on error management mechanisms. The thesis further develops practical methods to exploit these vulnerabilities and assess their impact on the application and system levels.

To validate our findings, a simulated environment closely replicating a real-world fieldbus network within an ICS is constructed. The open-source PyProfibus library is extended to incorporate a PROFIbus Device stack and implement essential PROFIsafe features, enabling the testing of exploits in a controlled setting. The experimental results demonstrate the feasibility and potential impact of the developed attacks, showcasing how vulnerabilities in error management mechanisms can be exploited to disrupt communication, manipulate data, and even gain unauthorized control of industrial processes.

The main contributions of this thesis to the field of ICS security are:

- **Reparameterization Attack:** The thesis introduces a novel attack strategy that exploits the reparameterization vulnerability in the PROFIbus protocol, enabling an attacker to gain unauthorized control of a Device and disrupt the entire network.
- **CRC Manipulation:** The thesis presents an enhancement to an existing attack on the PROFIsafe protocol, demonstrating how to efficiently manipulate the CRC2 checksum to bypass safety mechanisms and potentially compromise critical systems.

Countermeasures for UART, PROFIbus, and PROFIsafe:

The thesis proposes concrete countermeasures to address the identified vulnerabilities at different layers of the protocol stack. These include:

- The introduction of arbitration mechanisms at the UART/RS-485 level to prevent unauthorized access and data manipulation.
- The use of static parameters in PROFIbus to mitigate the risk of reparameterization attacks.

- The implementation of frequent random key exchanges in PROFIsafe to enhance the security of the CRC mechanism and protect against unauthorized access.

These contributions collectively enhance the understanding of security challenges in ICS and provide actionable insights for safeguarding critical infrastructure against cyber threats.

2 | Background

This chapter provides the foundational structures and concepts necessary for understanding the context of this work and its subsequent results.

2.1. Industrial Control Systems

Industrial Control Systems (ICS) are the backbone of modern industrial operations, supporting critical infrastructure and manufacturing processes worldwide. These complex systems enable the automation, monitoring, and control of industrial processes, ensuring efficiency, safety, and productivity.

ICS are responsible for executing a wide array of functions in industrial environments. These functions include:

- Process Control: ICS regulate and maintain process variables such as temperature, pressure, flow rate, and level within desired parameters. This ensures consistent product quality, safety, and operational efficiency.
- Data Acquisition and Monitoring: ICS collect real-time data from sensors and field devices, providing operators with valuable insights into the status of industrial processes. This data is used for monitoring, analysis, and decision-making.
- Supervisory Control: ICS enable remote monitoring and control of industrial processes from centralized control rooms. Operators can adjust setpoints, issue commands, and respond to alarms or abnormal conditions.
- Safety Systems: ICS incorporate safety instrumented systems (SIS) designed to detect and mitigate hazardous situations. SIS can automatically shut down processes, isolate equipment, or initiate safety procedures to protect personnel and assets.

Real-world examples of ICS applications include:

- Manufacturing: ICS control production lines, robotic systems, and material handling equipment in factories.

- Energy: ICS manage power generation, transmission, and distribution networks, ensuring the stability and reliability of the electrical grid.
- Water Treatment: ICS automate water treatment processes, monitoring water quality and adjusting chemical dosages to meet regulatory standards.
- Transportation: ICS control traffic signals, train systems, and airport operations, enhancing safety and efficiency.

2.1.1. Layers of ICS

ICS are typically organized into a hierarchical structure consisting of multiple layers:

- Field Level: This layer encompasses the physical devices and sensors that directly interact with the industrial process. It includes sensors, actuators, motors, valves, and other field devices.
- Control Level: This layer houses programmable logic controllers (PLCs), which are the brains of ICS. PLCs execute control logic, process data from field devices, and issue commands to actuators.
- Supervisory Level: This layer provides a human-machine interface (HMI) for operators to monitor and control the industrial process. It includes SCADA (Supervisory Control and Data Acquisition) systems, which visualize process data, generate alarms, and facilitate remote control.
- Enterprise Level: This layer integrates ICS with higher-level business systems such as enterprise resource planning (ERP) and manufacturing execution systems (MES). It enables data exchange and collaboration between different departments within an organization.

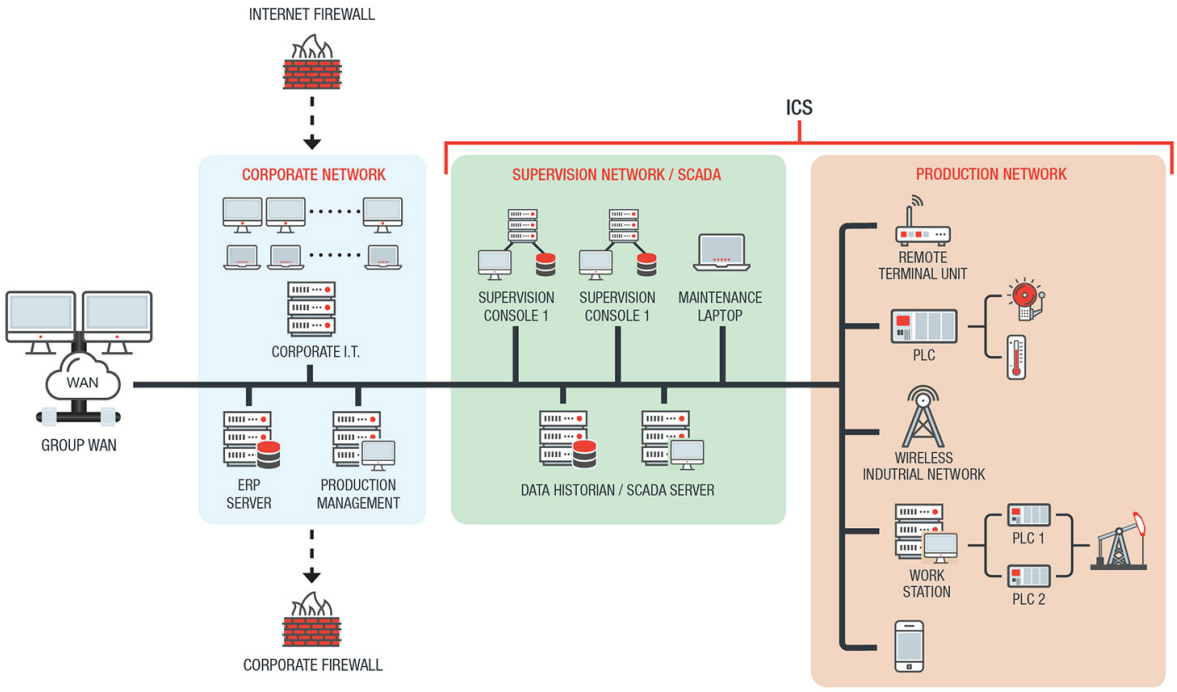


Figure 2.1: Industrial Control System overview

2.1.2. PLCs and Industrial Networks

Programmable Logic Controllers are essential components of ICS. They are industrial computers designed to execute control logic in harsh environments. PLCs continuously monitor inputs from field devices, make decisions based on pre-programmed logic, and send outputs to control actuators.

PLCs communicate with field devices and other PLCs over industrial networks. These networks can be wired or wireless and use various protocols such as Ethernet/IP, Modbus, PROFIbus, and PROFInet. Industrial networks enable real-time data exchange, remote monitoring, and control of industrial processes.

2.2. Fieldbus

Fieldbus is a family of industrial communication protocols designed for real-time distributed control within Industrial Control Systems, including protocols like PROFIBUS, Modbus RTU, DeviceNet, and AS-Interface. It serves as the nervous system of an industrial plant, enabling seamless communication between field devices (sensors, actuators) and higher-level controllers (PLCs). Unlike traditional point-to-point wiring, fieldbus utilizes a multi-drop topology, where multiple devices share a single communication line,

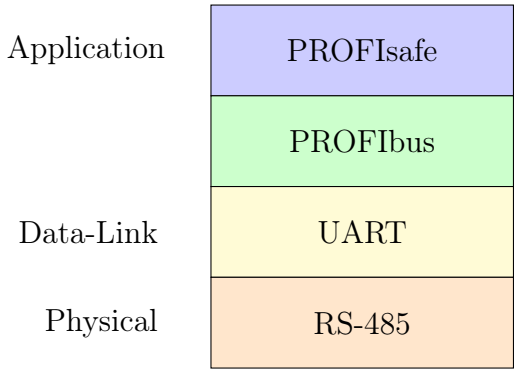
reducing cabling costs and simplifying network architecture.

The main advantages of these protocols are:

- **Reduced Cabling:** The multi-drop topology significantly reduces the amount of cabling required compared to point-to-point connections, leading to cost savings and easier installation.
- **Real-Time Communication:** Fieldbus protocols are optimized for real-time communication, ensuring that control signals and data are transmitted and received with minimal latency, crucial for critical control applications.
- **Diagnostics and Maintenance:** Many fieldbus protocols support advanced diagnostics, allowing for easier identification and troubleshooting of network and device issues, improving maintenance efficiency.

2.3. PROFIbus

PROFIbus (Process Field Bus [1, 2, 4–6]) is the most adopted fieldbus communication standard in the realm of industrial automation. Engineered for robust and reliable data exchange between devices in diverse industrial environments, PROFIbus has become a cornerstone of modern manufacturing and process control systems.
PROFIbus requires a fixed stack of protocols that have to be implemented in the industrial network, and that we are going to describe in this chapter:



2.3.1. Key Characteristics

- Open Standard: PROFIbus adheres to international standards, ensuring interoperability between devices from different manufacturers.
- Real-Time Communication: PROFIbus supports real-time data exchange, which is

crucial for time-sensitive control applications in industrial automation.

- Multi-Controller Capability: PROFIbus allows multiple Controllers to coexist on the same network, enabling distributed control architectures (but a Device is always controlled by one Controller at a time).
- Cyclic and Acyclic Communication: PROFIbus supports both cyclic data exchange for continuous process monitoring and control, as well as acyclic communication for event-driven data transfers and alarms.

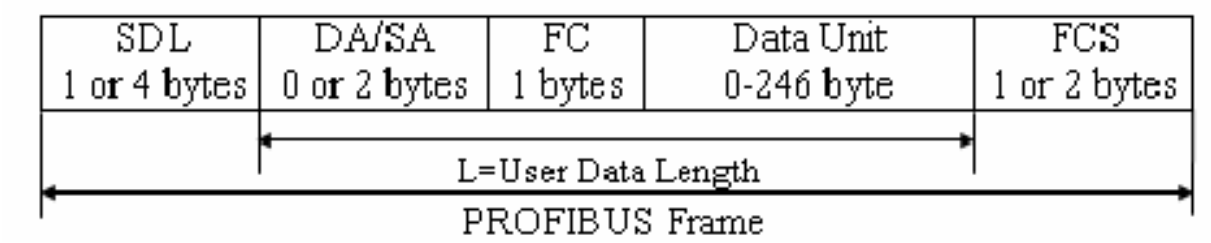


Figure 2.2: PROFIbus frame format

2.3.2. Frames

PROFIbus data is transmitted in frames, which consist of several fields:

- SD (Start Delimiter): Indicates the beginning of a frame. The specific value of the SD also distinguishes the type of telegram. There are four types of Start Delimiters (SD) for request and response telegrams, plus a fifth response for a short acknowledgment.
- LE (Length): Specifies the length of the user data field. In the case of variable data length telegrams (SD2 telegrams), the range is from DU (Data Unit) + 5 bytes to 249 bytes.
- DA (Destination Address): Identifies the intended recipient of the frame.
- SA (Source Address): Identifies the sender of the frame.
- FC (Function Code): Specifies the type of data or command contained in the frame. It is used to identify the type of telegram, such as request, acknowledgment, or response telegrams.
- DU (Data Unit): Contains the actual user data being transmitted. The length of the data field is variable, up to 244 bytes, but is typically limited to 32 bytes.

- FCS (Frame Check Sequence): Used for error detection. It is calculated as the sum of the ASCII bytes of information from the DA (Destination Address) field to the end of the DU (Data Unit) field, modulo 256.
- ED (End Delimiter): Indicates the end of a frame and has a fixed value of 0x16 (hexadecimal).

2.3.3. Devices

PROFIbus communication adheres to a Controller-Device architecture:

- Controller: The Controller device initiates communication and controls the flow of data on the bus. It sends requests to Devices and receives responses. In PROFIbus DP, there are two classes of Controllers:
  - Class 1 Controller: Handles normal communication and data exchange with assigned Devices.
  - Class 2 Controller: Primarily used for commissioning Devices and diagnostic purposes (often substituted by configuration tools).
- Devices: Device devices respond to requests from the Controller, providing data or executing commands as instructed. They have no bus access rights and can only acknowledge received messages or send response messages upon request.

2.3.4. Cyclic Communication

Cyclic communication is a core operational mode in PROFIbus. The Controller periodically polls the Devices, requesting process data and sending control commands. This cyclic exchange ensures continuous monitoring and control of industrial processes. The order in which the Devices are queried stays the same, and if a Device does not answer, the Controller will take care of keeping the communication with the other devices while trying to fix the problems.

2.3.5. Error Management

PROFIbus employs several mechanisms for error detection and handling:

- Frame Check Sequence (FCS): Each frame includes an FCS for error detection at the receiver side.
- Timeout Monitoring: Controllers monitor response times from Devices. If a Device

fails to respond within a specified timeout period, the Controller can initiate error recovery procedures. In a similar fashion, if the Device does not receive any request before its watchdog expires, it will assume to not be controlled anymore and goes to a safe state.

- Clear Mode: A PROFIbus Controller operates in two modes: Operate and Clear. In Clear Mode, the Controller sends zero-length data or data set to 0. Devices that support fail-safe mode can respond by clearing their outputs or assuming a predefined fail-safe state. This mechanism helps to ensure a safe and predictable state in case of communication issues or Controller failure.

2.4. PROFIsafe

The primary goal of PROFIsafe [3] is to enable the safe and reliable transmission of safety-related data over standard industrial networks (PROFIbus and PROFInet).

PROFIsafe is utilized across a range of industrial environments where functional safety is critical. These include:

- Factory Automation: In manufacturing plants, PROFIsafe ensures the safe operation of machinery and processes. For instance, it can be used to transmit emergency stop signals, monitor safety interlocks on robotic systems, or control safety barriers on production lines.
- Process Automation: In process industries like chemical plants or oil refineries, PROFIsafe enables the safe control of critical processes. It can be used to transmit signals from safety sensors, monitor the status of safety valves, or control emergency shutdown systems.
- Machinery: PROFIsafe is also employed in various machinery applications, such as packaging machines, cranes, or elevators, to ensure the safe operation of these machines. It can be used to transmit safety-related signals, monitor safety sensors, or control safety actuators.

To achieve safety in communication, PROFIsafe employs four key mechanisms:

- Virtual Monitoring Number (MNR): The MNR is a sequence counter used to ensure the authenticity and correct order of transmitted safety messages. It helps detect issues like unintended repetition, incorrect sequence, loss, or insertion of messages.
- Watchdog Time Monitoring with Acknowledgment: Both the sender and receiver of safety messages maintain watchdog timers. The sender expects an acknowledgment

from the receiver within a specified time, and if this acknowledgment is not received, it indicates a potential communication failure, triggering a safe response.

- Codename per Communication Relationship: Each safety communication relationship between devices is identified by a unique codename. This codename is used for authentication purposes and helps prevent unauthorized access or manipulation of safety data.
- Cyclic Redundancy Checking (CRC) for Data Integrity: CRC is used to detect errors in the transmitted data. The sender calculates a CRC value based on the message content and includes it in the message. The receiver recalculates the CRC and compares it with the received value. If they don't match, it indicates data corruption, triggering a safe response.

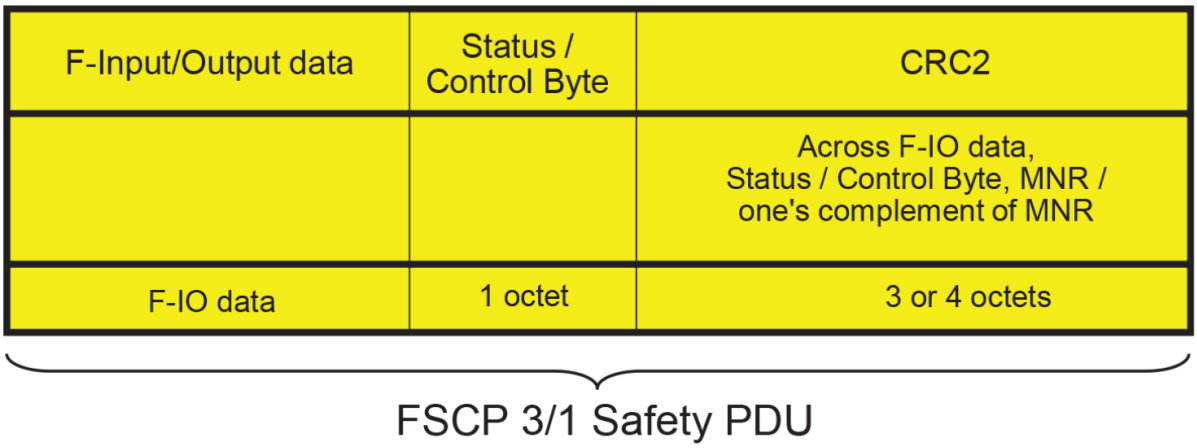


Figure 2.3: PROFIsafe frame format

2.5. UART

The Universal Asynchronous Receiver-Transmitter (UART) protocol is a fundamental method for serial communication between devices. It enables the transmission of data one bit at a time over a single wire, making it simple and cost-effective for connecting various electronic components.

2.5.1. Key Characteristics of UART

- Asynchronous Communication: UART doesn't require a shared clock signal between the sender and receiver. Instead, it relies on start and stop bits to frame each data byte, allowing for flexibility in timing.

- Full Duplex: UART supports simultaneous transmission and reception of data, enabling efficient two-way communication.
- Simple Hardware: UART implementation requires minimal hardware, making it a popular choice for embedded systems and low-cost devices.
- Versatile: UART is widely used in various applications, including computer peripherals, industrial control systems, and communication between microcontrollers.

2.5.2. Relationship with RS-485 and PROFIbus

UART can be used in conjunction with other standards to enable more complex communication systems.

- RS-485: UART signals can be converted to RS-485 differential signals using dedicated transceiver chips. This allows UART-based devices to communicate over longer distances and in noisy environments, leveraging the benefits of RS-485's robustness.
- PROFIbus: PROFIbus is a fieldbus protocol that utilizes RS-485 as its physical layer. UART, at the data-link layer, is used as interface between the two, enabling communication within industrial automation systems.

2.5.3. Frames and Communication

UART data is transmitted in frames, each consisting of:

- Start Bit: A single bit with a logic low level, indicating the beginning of a frame.
- Data Bits: Typically 5 to 9 bits, representing the actual data being transmitted.
- Parity Bit (Optional): An additional bit used for error detection.
- Stop Bits: One or two bits with a logic high level, signaling the end of a frame.

The sender and receiver must agree upon the number of data bits, the presence of a parity bit, and the number of stop bits before communication can occur.

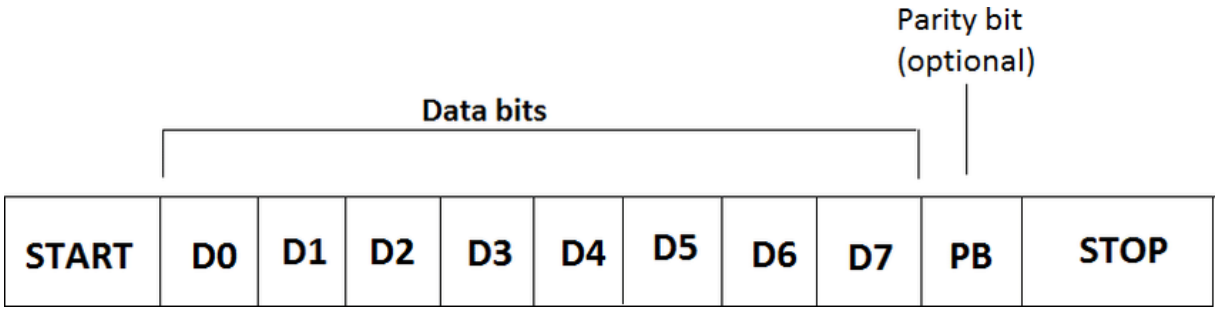


Figure 2.4: UART frame format

2.5.4. Error Management

UART has limited built-in error detection mechanisms. The optional parity bit can detect single-bit errors, but more sophisticated error detection and correction techniques are often implemented at higher protocol layers.

Common UART error conditions include:

- Framing Errors: Occurwhen the receiver fails to detect a valid start or stop bit.
- Overrun Errors: Occur when the receiver's buffer overflows because it cannot process incoming data fast enough.
- Parity Errors: Occur when the received parity bit doesn't match the calculated parity for the data bits.

To mitigate these errors, applications often employ techniques like checksums, cyclic redundancy checks (CRC), or retransmission of data.

2.6. RS-485

RS-485 (Recommended Standard 485) is a widely used electrical standard for serial communication in industrial and automation settings. It defines the electrical characteristics of a balanced differential signaling system, enabling reliable data transmission over long distances and in electrically noisy environments.

2.6.1. Key Characteristics of RS-485:

- Balanced Differential Signaling: RS-485 utilizes two wires (A and B) for data transmission, where the signal is the voltage difference between the two. This differential signaling provides excellent noise immunity, as common-mode noise is rejected.

- Multipoint Topology: RS-485 allows multiple devices (up to 32) to be connected on the same bus, creating a multipoint network topology.
- Long Distance Transmission: RS-485 supports communication distances up to 1200 meters at lower data rates and shorter distances at higher data rates.
- High Speed: RS-485 can achieve data rates up to 10 Mbps over shorter distances.
- Robustness: RS-485 is designed to operate in harsh industrial environments, with resistance to electrical noise and interference.

2.6.2. Electrical Functioning

RS-485 operates using differential voltage levels. A logic '1' is represented by a positive voltage difference between A and B, while a logic '0' is represented by a negative voltage difference. The voltage levels are typically around +200 mV for a logic '1' and -200 mV for a logic '0'.

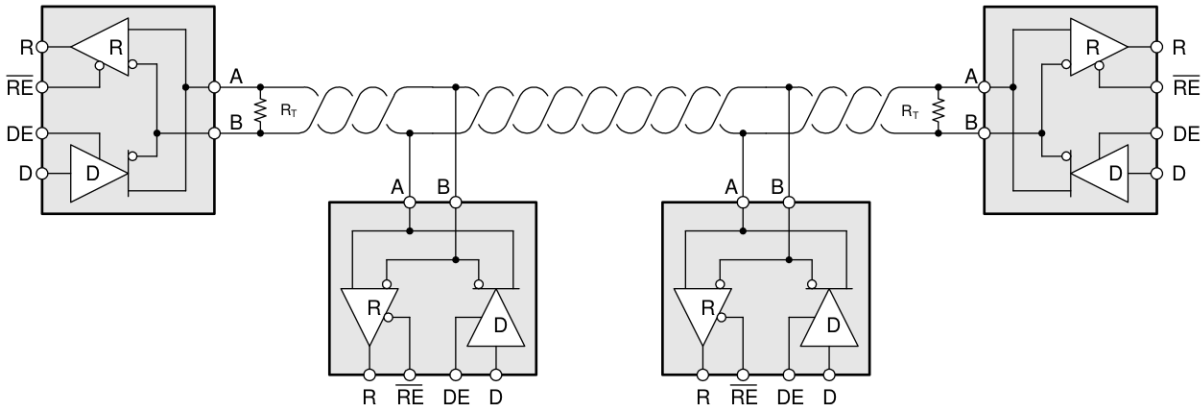


Figure 2.5: RS-485 standard

2.7. Safety & Security

The standard highlights that traditional industrial communication protocols, such as UART, PROFIbus, and even PROFIsafe itself, were primarily developed with a focus on safety rather than security. The distinction between these two concepts is crucial:

- Safety pertains to protecting systems from unintentional harm or malfunctions. In the context of industrial communication, safety protocols aim to prevent accidents, equipment damage, or injuries by ensuring the reliable and error-free transmission of critical control signals.

- Security, on the other hand, involves safeguarding systems from intentional attacks or unauthorized access. Security measures focus on preventing malicious actors from disrupting operations, stealing data, or causing harm.

The increasing connectivity and complexity of modern industrial networks have made them more vulnerable to cyberattacks, highlighting the need for enhanced security measures in addition to traditional safety protocols.

3 | Motivation

3.1. Problem Statement

Industrial Control Systems (ICS) are fundamental to modern industrial operations, enabling efficient and automated production across various sectors. However, these systems, often organized as Supervisory Control and Data Acquisition (SCADA) systems, have become prime targets for malicious actors seeking to disrupt production, inflict financial losses, and even endanger human lives.

Several factors contribute to the complexity of securing ICS networks. First, the protocols underpinning these systems, particularly at the lower levels, were developed in an era where security was not a primary concern. Protocols like Modbus and PROFIbus, which is central to this thesis, lack robust authentication and encryption mechanisms, making them vulnerable to exploitation. Second, the security requirements of lower-level cyber-physical systems (CPS) diverge from those of traditional IT systems. As Huang et al. [15] highlight, availability takes precedence over confidentiality in CPS due to the critical need to maintain continuous operation. Historically, the isolation of these systems within internal networks has fostered a false sense of security, further exacerbating the challenge.

Real-world attacks have underscored the disruptive potential and devastating impact of SCADA system breaches. Fauri et al. [11] consider three notable examples:

- **Stuxnet:** This sophisticated malware targeted Iranian nuclear enrichment facilities, manipulating specific centrifuges to operate at destructive speeds, rendering them unusable. An estimated 20% of Iran's centrifuges were compromised. The targeted programmable logic controllers (PLCs) were from Siemens and utilized PROFIbus, the same focus of this thesis.
- **Dragonfly:** This campaign primarily focused on industrial espionage and intellectual property theft rather than direct control of industrial processes. It specifically targeted industrial gateways and routers used in various sectors, including aviation, energy, pharmaceuticals, and food and beverage.

- **BlackEnergy:** This cyberwarfare attack targeted three Ukrainian power distribution utilities, causing a blackout for several hours. The attackers exploited the malware to seize control of SCADA workstations and human-machine interfaces (HMIs), issuing commands to open remote station breakers. Simultaneously, they deployed malicious firmware on gateway devices, disabling them and hindering recovery efforts.

While these attacks demonstrate the sophistication and devastating potential of adversaries, often backed by nation-state resources, this thesis challenges a more fundamental assumption: the perceived security of the Controller/Device protocol structure. The traditional hierarchy, where Devices are assumed to be passive recipients of commands, creates a false sense of security. We demonstrate how, even with relatively simple techniques, a compromised Device can subvert this hierarchy, gain control over other Devices, and effectively elevate itself to a Controller role, thereby undermining the entire network's integrity.

The vastness and heterogeneity of ICS networks, coupled with the increasing integration of wireless connections and the internet with traditional fieldbus networks, significantly expand the attack surface. Defending these systems is far from trivial. As Fauri et al. [11] explains, encryption alone may not be sufficient to prevent the aforementioned attacks and can be difficult to implement due to real-time processing constraints. Therefore, it's crucial to explore security measures at all levels of the network architecture, with particular attention to the foundational data link layer. This is the primary focus of this thesis, exploring the exploitation and mitigation of vulnerabilities at the data link layer, which can have far-reaching consequences for the entire system.

3.2. State of the Art

The increasing integration of digital technologies and network connectivity in industrial environments has amplified the attack surface and the potential for cyber threats to industrial control systems (ICSs). While these advancements in technology offer numerous benefits, such as improved efficiency and productivity, they also introduce new vulnerabilities that can be exploited by malicious actors. Research in this area is crucial for understanding these vulnerabilities and developing effective countermeasures to protect critical infrastructure. The communication stack under consideration comprises RS-485, UART, PROFIbus, and PROFIsafe. This selection is based on their widespread adoption in ICS fieldbus networks, particularly in safety-critical systems where the robustness and industry support for PROFIsafe are highly valued.

3.2.1. Attacks on UART and RS485

The UART (Universal Asynchronous Receiver/Transmitter) protocol and RS485 standard are widely used in industrial control systems (ICS) for communication between devices, and they form the lowest level of the PROFIbus stack. However, their inherent vulnerabilities can be exploited by attackers to disrupt or manipulate industrial processes.

UART-Based Attacks Many IoT and embedded devices, which often utilize UART for communication, lack robust security mechanisms and are vulnerable to attacks that exploit their hardware interfaces. These attacks can range from capturing boot logs to brute-force attacks on login credentials. Mishra et al. [18] developed a hardware auditing testbed that combines software toolchains with low-cost community hardware to perform hardware auditing, enabling the identification of vulnerabilities related to UART communication. The testbed can perform various automated tasks, such as signature-based scans, identifying UART consoles, investigating firmware, analyzing bootloader logs, and detecting hardcoded secrets. The recent integration of IoT in PROFIbus systems, and the resulting increase in attack surface, makes also this channel relevant in the security scenario.

RS485-Based Attacks Ochiai et al. [22] define five types of field-bus attacks:

- Spoofing: An attacker transmits jamming signals on the communication line after reading the first bytes of a request, breaking the later part of the frame. Then, the legitimate server receives a broken message and does nothing. Instead, the attacker responds its spoofed message.
- Random Collision: An attacker transmits jamming signals on the communication line at random when it observes a legitimate signal. It can deny communications and the fieldbus operation will be suspended.
- Horizontal Scan: An attacker requests Servers on the network by changing the Server IDs, expecting some responses. This attack can steal the network configuration.
- Vertical Scan: An attacker requests to read from a wider range of register addresses one by one. This attack can explore data which are usually not used and exchanged on the communication line.
- Evil Twin: An attacker sends requests on behalf of the legitimate Client. The requests can be identical to those of the legitimate Client. This attack can override register values on the Server without being noticed.

These attacks exploit vulnerabilities in the RS-485 standard to manipulate data, disrupt communication, or gain unauthorized access to the network, also implying relevant effects at the application level. The researchers developed datasets using a real-world Modbus/RS-485 testbed, capturing the analog signals of the communication line during both benign and malicious activities. By analyzing these signals using machine learning algorithms, they were able to achieve high accuracy in detecting and classifying these attacks, with some limitations dictated by the attacker's behavior.

Electromagnetic Interference (EMI) Attacks Intentional electromagnetic interference (IEMI) can be used to manipulate serial communication data wirelessly, including UART and RS485. There are two types of attack waveforms: simple and complex, both capable of inducing bidirectional bit flips in UART and I2C communication systems. The simple waveform is easier to generate but requires precise timing, while the complex waveform is more tolerant of timing errors but requires more knowledge of the targeted system. There are several passive countermeasures, such as twisted cables, coaxial cables, and fiber-optic transmission, to mitigate the impact of IEMI attacks. However, these techniques are unlikely to be employed in real-world attacks due to the extended physical presence required of the attacker, the specialized equipment necessary, and the comparatively weaker results yielded by such methods [10].

3.2.2. PROFIbus and other fieldbus attacks

PROFIbus lacks several key security features, including confidentiality, integrity checks, and authentication or authorization controls. The absence of these controls makes PROFIBUS susceptible to various attacks, such as message injection, eavesdropping, and replay attacks. Moreover the increasing integration of PROFIBUS networks with corporate networks and the internet further exacerbates these risks, as it exposes these systems to a wider range of potential attackers.

A good image of the security scenario is given by Krotofil et al. [16], observing that the increasing integration and connectivity of ICS has led to heightened security risks. They emphasize that the traditional focus on dependability and safety in ICS design needs to evolve to incorporate robust security measures, especially given the rising threat of cyberattacks. The authors highlight the gap between the security expectations of legacy systems and the economic feasibility of implementing advanced security solutions. They also stress the importance of understanding the physical and control aspects of ICS to design effective security mechanisms, introducing the concept of "secure control." The paper concludes by discussing the challenges and open questions in ICS security research,

such as the analysis of intrusion detection systems during abnormal situations and the accurate identification of the root cause of process disturbances.

Error Management Mechanisms A significant vulnerability in industrial communication protocols lies in their error management mechanisms. Watson et al. [28] analyzed these mechanisms, designed to ensure reliable communication in the face of errors, and explained how they can be exploited by attackers to disrupt or manipulate industrial processes. In the case of PROFIBUS, the lack of authentication and authorization controls allows attackers to inject malicious messages that can trigger error states and potentially lead to a denial-of-service (DoS) condition. This vulnerability is particularly concerning in safety-critical systems, where a DoS attack can have severe consequences.

SCADA Attacks Morris et al. [21] provide a widely used categorization of attacks on SCADA control systems. It is a comprehensive set of 17 attacks, divided into reconnaissance, response and measurement injection, command injection, and denial of service (complete classification in the table below). These attacks highlight the diverse range of vulnerabilities that can be exploited in industrial control systems, emphasizing the need for a multi-layered security approach. They demonstrate the feasibility of these attacks in a laboratory setting, using commercial control system hardware and software to model a gas pipeline and a water storage tank. The tests are performed exploiting vulnerabilities of the Modbus RTU protocol, but most of them, due to lack of authentication and encryption, can be replicated on Profibus. Their findings underscore the importance of understanding the potential impact of cyberattacks on critical infrastructure and the need for robust security measures to mitigate these risks.

Attack Index	Name	Classification
1	Address Scan	Reconnaissance
2	Function Code Scan	Reconnaissance
3	Device Identification	Reconnaissance
4	Naïve Read Payload Injection	NMRI
5	Invalid Read Payload Size	NMRI
6	Naïve False Error Response	NMRI
7	Sporadic Sensor Measurement Injection Attack	NMRI
8	Slope Sensor Measurement Injection	CMRI
9	High Slope Measurement Injection	CMRI
10	High Frequency Measurement Injection	CMRI
11	Altered System Control Scheme	MSCI
12	Altered Actuator State	MSCI
13	Altered Control Set Point	MPCI
14	Force Listen Only Mode	MFCI
15	Restart Communication	MFCI
16	Invalid Cyclic Redundancy Code (CRC)	DOS
17	MODBUS Slave Traffic Jamming	DOS

Figure 3.1: SCADA attacks classification (Morris)

Another important research on SCADA is from Rubio-Hernan et al. [25], that scrutinizes the security of a watermark-based detection scheme that was originally proposed by Mo and Sinopoli [19] and Mo et al. [20] for the purpose of identifying attacks targeting cyber-physical systems. The work introduces a new adversary model, referred to as the "cyber-physical adversary," and proceeds to demonstrate that the original scheme is susceptible to attacks perpetrated by this type of adversary. In response, the authors put forth an enhanced detection scheme that is founded on a multi-watermark authentication signal, showcasing its efficacy in effectively detecting both cyber and cyber-physical adversaries. The performance of this proposed scheme is then rigorously validated through numerical

simulations.

Off-the-Shelf Attack Tools Another threat is posed by off-the-shelf offensive solutions, both hardware and software, for penetrating industrial control systems (ICS). Attackers can exploit social engineering tactics, such as leaving a malicious USB device (like the Hak5 Rubber Ducky) in a location where an employee might find and plug it into a computer connected to the ICS network. Once plugged in, the device can automatically execute scripts to establish a reverse shell connection to the attacker's machine, allowing them to remotely access and control the compromised computer. The attacker can then leverage this access to move laterally within the network and potentially compromise other devices, including PLCs and HMIs. It is demonstrated how these readily available tools can be used to gain access to unprotected industrial plants and carry out various attacks, such as Man-in-the-Middle (MITM) and Legal-Client-to-Server (LCSA) attacks. Hotellier et al. [14] successfully manipulated the setpoint of a pressure regulator in a compressor station using a combination of social engineering and off-the-shelf attack tools.

3.2.3. PROFIsafe Attacks

Even modern protocols like PROFIsafe, designed for safety-critical systems, have been found to have vulnerabilities. It is possible to manipulate safety-related process data in PROFIsafe without detection due to the lack of robust authentication mechanisms. The increasing levels of vertical and horizontal integration in industrial automation further exacerbate these security risks. While PROFIsafe was designed with safety in mind, adhering to the "black channel" principle, which assumes that the underlying transmission system is secure, Akerberg et al. [30] findings challenge this assumption. Their research demonstrates a successful man-in-the-middle attack where they manipulate safety data by recalculating the Cyclic Redundancy Check (CRC) checksum.

To carry out their attack, they first had to "break the code" of the PROFIsafe protocol. This involved obtaining the current Virtual Consecutive Number (VCN) and the static CRC1 checksum. They achieved this by receiving a safety container and using brute force to calculate all valid combinations of CRC1 and VCN that would generate the same CRC2 checksum as the received message. By analyzing subsequent frames and discarding invalid combinations, they were able to determine the correct CRC1 and VCN values. Once these values were obtained, they could modify the safety-related I/O data within the PROFIsafe containers and recalculate the CRC2 checksum, effectively bypassing the safety measures of the protocol. This allowed them to change the safety-related process data without detection, demonstrating a significant vulnerability in the PROFIsafe protocol.

Protocol Attacks

Protocol	Attacks
RS-485 & UART	- Spoofing [22]
	- Random Collision [22]
	- Horizontal Scan [22]
	- Vertical Scan [22]
	- Evil Twin [22]
	- EMI [10]
PROFIbus	- Error Management Abuse [28]
	- Reconnaissance [21]
	- Response Injection [21]
	- Command Injection [21]
	- Denial of Service [21]
	- Off the Shelf Tools [14]
PROFIsafe	- CRC Manipulation [30]

Table 3.1: Protocols and their associated attacks.

3.2.4. Countermeasures

Intrusion Detection Systems The vulnerabilities in both legacy and modern protocols have led to the emergence of specific attack techniques targeting industrial bus networks. Li et al. [17] focus on the PROFIbus-DP protocol, discussing fault injection attacks that involve injecting non-compliant messages and semantic attacks that exploit legitimate protocols and parameters to manipulate industrial processes. They propose DpGuard, an attack detection system that leverages finite-state machine (FSM) models to identify and mitigate such attacks. However, the researchers' claims of high accuracy (100% for fault injection and 99.8% for semantic attacks) should be interpreted with caution. The formal definition of the FSM and the algorithms used appear to be imprecise, potentially leading to a high rate of false positives, common for anomaly-based IDSs. For instance, any network anomaly (noise, EMI, voltage fluctuations, etc.) could be misinterpreted as a fault injection attack, a critical issue not adequately addressed in their testing. Moreover, the training and testing methodology is limited, with the system trained on a relatively small dataset (20,000 packets) and tested on an even smaller one (6288 packets), both lacking real-world error scenarios, thus inflating the reported accuracy. These limitations raise concerns about the practical effectiveness of DpGuard in

real-world industrial environments.

Other research on the detection of fault injections has focused on sensor data injection attacks, for example from Hotellier et al. [14], based on the transmission of TPDOs (Transmitted Process Data Objects). This transmission is periodic in standard behavior, but since a false sensor data injection attack would disrupt this periodicity, the researchers created a new pattern called Minimal Period to detect this type of attack. The Minimal Period pattern is defined as the "amount of time in which a state formula has to hold at most once". In other words, it ensures that a TPDO is not transmitted more than once within a given period.

Zhou et al. [29] propose a security model tailored for PROFIbus industrial networks and develop an embedded gateway designed to facilitate secure, real-time monitoring of PROFIbus devices over the Internet. This gateway integrates a suite of security measures, including protocol conversion, identity authentication, access control, and data encryption, all aimed at safeguarding against unauthorized access and potential data breaches.

In general, the detection of fault injections at the real-time level (e.g., Profibus) or physical level (e.g., UART) of the architecture is extremely challenging due to the high number of false positives. As a result, the best solutions, where present, are often specific to a particular attack, and there is not yet a general solution for this category of attacks.

Intrusion Tolerance The security challenges associated with the integration of cyber and physical systems in Industry 4.0 and 5.0 are increasing. Huang et al. [15] propose a multi-layer cyber-security protection architecture for networked industrial processes, emphasizing the importance of intrusion tolerance at the real-time control layer. The authors argue that traditional IT security solutions are not sufficient for industrial CPSs due to their unique characteristics and requirements. Their proposed approach involves a closed-loop defense framework that combines intrusion detection, impact assessment, strategy decision, and intrusion response.

Combined Safety and Security Risk Analysis Reichenbach et al. [23] propose an approach that integrates safety and security risk assessments by incorporating the Safety Integrity Level (SIL) from the IEC 61508 standard into the Threat Vulnerability and Risk Assessment (TVRA) framework. The SIL, which ranges from 1 to 4, indicates the level of safety integrity required for a safety function. The higher the SIL, the more stringent the safety requirements and the lower the tolerance for failure.

In the combined approach, the SIL is treated as an additional factor that influences the

impact of a security threat. The TVRA method calculates the overall risk by considering the likelihood of a threat occurring and the potential impact if it does. By including the SIL in the impact calculation, the approach ensures that threats targeting high-SIL safety functions are assigned a higher impact, reflecting the potentially severe consequences of a security breach affecting critical safety systems.

The integration of SIL into the TVRA framework allows for a more detailed and comprehensive risk assessment that considers both safety and security perspectives. This enables organizations to prioritize threats and vulnerabilities based on their potential impact on both safety and security, leading to more effective risk mitigation strategies. The approach also promotes a more complete understanding of the interplay between safety and security, encouraging organizations to address both aspects in a coordinated and integrated manner.

An interesting taxonomy is presented by Tomur et al. [26]. They focus on threats that specifically target the safety functions within Industrial IoT applications, and they proceed to identify potential attack scenarios that could have direct consequences for both physical assets (including human life) and cyber assets (like the availability of Industrial IoT-based manufacturing systems). They also provide mitigations to the threats, for example using the 5G integrity features to defend safety critical messages from tampering, extending the Reichenbach's ideas to a different attack surface.

3.3. Goals and Challenges

The primary aim of this thesis is to investigate and exploit vulnerabilities related to error management within industrial communication protocols, specifically UART, PROFIbus, and PROFIsafe. By comprehensively understanding the weaknesses inherent in these protocols, we develop novel attack strategies that can compromise the integrity and availability of industrial systems.

Our objectives are:

1. **Identification and Analysis:** Conduct a comprehensive identification and analysis of both theoretical and practical vulnerabilities present within the data-link layer (UART) of industrial networks. This will involve a thorough examination of relevant standards, common implementations, and a specific focus on error management issues.

2. **Practical Exploitation and Impact Assessment:** Develop practical methods for exploiting the identified vulnerabilities and assess their impact at both the ap-

plication and system levels. This includes the creation of novel attack techniques to illustrate the potential consequences of these exploits.

3. **Countermeasure Development:** Define and propose countermeasures designed to prevent the execution of the developed attacks and potentially mitigate similar exploits. The goal of these countermeasures is to enhance the overall security of industrial communication systems.

In pursuit of these goals, we encountered and addressed several challenges. The first challenge was the need to conduct an extensive study of protocol standards, vendor manuals and implementations, and industrial communication and safety standards (IEC 61158 and IEC 61784) in order to establish a comprehensive understanding of the security aspects of the analyzed systems. The second challenge arose from the fact that the majority of PROFIbus and PROFIsafe implementations are proprietary and closed source. To facilitate the testing of our exploits, we extended the open-source library "pyprofibus", which provides a PROFIbus Controller stack, by augmenting it with a PROFIbus Device stack and implementing essential PROFIsafe features.

4 | Threat Model

In this chapter we are going to define the threat model, outlining the potential attack vectors and exploits that could be leveraged to compromise the chosen systems, setting the stage for a deeper exploration of the associated risks and countermeasures in the chapters that follow.

This threat model prioritizes the networks within Industrial Control Systems (ICS), particularly those utilizing fieldbus technology with PROFIbus and PROFIsafe protocols for communication between Programmable Logic Controllers (PLCs). This decision is firmly grounded in market data. According to HMS data [12, 13], until 2017 PROFIbus reigned as the dominant industrial network protocol, commanding a substantial 18% of the total market share for new installed nodes in 2015 alone .

While its growth has moderated due to the increasing adoption of Industrial Ethernet, PROFIbus remains the most widely implemented fieldbus protocol. It currently maintains a stable 7% market share of new installed nodes, a figure that has held steady for the past three years. Given its dominant position until recent years, it can be confidently asserted that PROFIbus remains the most prevalent protocol in industrial settings today.

Looking ahead, PROFIbus is expected to retain a significant market share, particularly in safety-critical environments. This is due to its "intrinsically safe" features, which make it well-suited for areas with fire and explosion risks. In such scenarios, PROFIsafe is frequently implemented as an additional layer of protection over PROFIbus.

4.1. Scope and Applicability

The vulnerabilities and exploits identified in this analysis target the data-link layer of PROFIbus and PROFIsafe protocols. Any industrial network using these protocols is susceptible to the described threats. The attacks exploit vulnerabilities in the UART protocol, making them potentially relevant to other systems using UART, such as Modbus.

In complex and wide environments, several tools can be used to get better control over the system, like HMI and SCADA softwares. They are not the primary focus of this

threat model, since they can communicate only with the network Controller, lacking the capability to directly control Devices or mitigate the attacks discussed. However, it's worth noting that compromising these systems could increase an attacker's potential to gain control over the Controller, though this scenario is not of interest for this analysis, because the assumption of being the Controller is way stronger than controlling a Device, and the attacks would became trivial due to the high control that the attacker already has over the network.

Importantly, this threat model is applicable to hybrid networks where PROFIbus is used within a larger network employing different protocols, such as Industrial Ethernet or wireless communication.

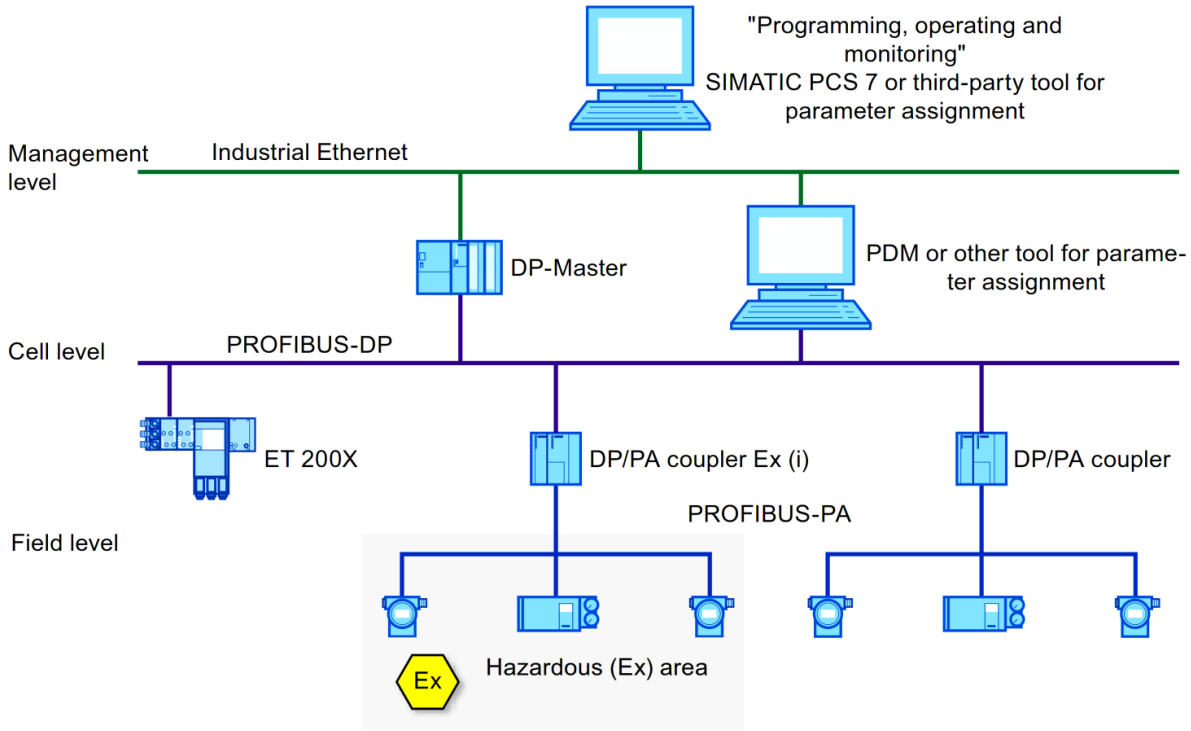


Figure 4.1: Typical network topology of a ICS with PROFIbus

4.2. Assets and Threat Agents

4.2.1. Assets

While ICSs contain many valuable assets, this analysis focuses on those directly vulnerable to the identified threats. These assets go beyond the immediate company environment, potentially impacting broader aspects like the supply chain.

1. **Essential Service Infrastructure:** Some ICSs play a pivotal role in the functioning of essential services, such as Smart Grids and Nuclear Plants. Disrupting these services, even through a denial-of-service attack, can have devastating consequences for society. These assets have been targets in real-world attacks, particularly those involving cyber warfare.
2. **People Safety:** Attacks exploiting the vulnerabilities identified can directly threaten human lives. Malicious commands to industrial robots or triggering explosions in critical systems can endanger personnel and potentially the general public.
3. **Industrial Production:** A prolonged downtime in production due to a cyberattack can lead to significant financial losses for companies.
4. **Machinery Integrity:** Malicious control of machinery, particularly actuators connected to PLCs, can lead to their destruction or compromise, as demonstrated by the Stuxnet attack [11].
5. **PLC and Industrial Network Availability:** Attacks can compromise the availability of PLCs and the entire industrial network. Some vulnerabilities enable attackers to render components inaccessible to their Controller or HMI, directly impacting network availability.
6. **Reputation:** Successful attacks can damage a company's reputation, and in the case of essential service infrastructure, even a nation's credibility.

4.2.2. Threat Agents

1. **Nation-State Actors:** Motivated by espionage, sabotage, geopolitical objectives, or wartime advantages, these actors possess advanced resources and capabilities to target critical infrastructure, disrupt essential services, and gain strategic leverage over adversaries. In conflict scenarios, these attacks can be employed to weaken an enemy's industrial base, create chaos, and disrupt military capabilities.
2. **Terrorist Groups:** Terrorist organizations can target ICSs to cause widespread disruption, fear, and economic damage.
3. **Cybercriminals:** Primarily driven by financial gain, they can target ICSs and employ extortion or data theft to get profit.
4. **Hacktivists:** Motivated by ideological or political agendas, they may target ICSs to raise awareness or disrupt operations to make a statement.
5. **Competitors:** Rival businesses may seek to gain an advantage by disrupting a

competitor's operations or stealing sensitive information through cyberattacks.

6. **Former Employees or Insiders:** Current or former employees with knowledge of the ICS and access credentials may pose a threat due to grievances, sabotage intentions, or financial incentives.

4.2.3. Mapping

Following the modeling of the primary assets and threat agents, we present a tabular mapping to clearly illustrate, for each asset, the attackers most likely to attempt its compromise.

	Nations	Terrorists	Cybercriminals	Hacktivists	Competitors	Employees
Essential Services	X	X	X	X		
People Safety	X	X			X	X
Industrial Production	X		X	X	X	X
Machinery Integrity	X		X	X	X	X
PLC and Network Availability			X	X	X	X
Reputation			X	X	X	X

Table 4.1: Threats to ICS by Different Actors

4.3. Assumptions

This study aims to provide a comprehensive framework for developing attacks and defining countermeasures within the prevalent industrial environment. To achieve this while maintaining a focused scope, certain assumptions have been made. These assumptions, with the exception of "Control a Device", should be viewed as directional guidelines rather than limitations. The vulnerabilities discussed may have broader implications beyond these assumptions, but exploring those is outside the scope of this thesis.

The proposed and executed attacks, however, are specifically tailored to the environment defined by the established assumptions.

- **PROFIbus and PROFIsafe:** This research focuses on the PROFIbus and PROFIsafe protocols, which are the foundational assumptions for our targeted system. PROFIbus is a widely used industrial protocol, especially for fieldbus networks, making it a relevant target. The use of PROFIbus implies the use of the RS-485 standard and the UART protocol at the data-link layer. For attacks on critical systems using PROFIsafe, we assume it's implemented on top of PROFIbus, not PROFInet.
- **Control a Device:** This is the only assumption that may be challenging for an attacker. All attacks developed in this work initiate with control over a PLC. However, this assumption is relatively weak compared to other research: it merely requires control of any Device PLC within the target PLC's network, not necessarily the target itself. Several methods can be employed to gain control:
  - **Network-Based Attacks:** These methods are fully remote and are based on the exposure of a PLC in external networks. This requirement is more common than is usually thought, because PLCs are complex devices that can be connected to several other devices (not only PROFIbus!):
    - **Sensors and Actuators:** PLCs are necessarily connected to at least one sensor or actuator. These devices, essential to PLC functionality, may have internet-facing features or utilize wireless connections, providing potential entry points for attackers to compromise the connected PLC.
    - **Advanced Features:** PLCs with remote access or diagnostic capabilities often require internet exposure, creating opportunities for exploitation via techniques like Man-in-the-Middle attacks.
    - **Hybrid Networks:** Complex, multi-layered ICS networks often employ diverse protocols, with higher levels frequently requiring wireless and in-

ternet connectivity due to long distances. Wang et al. [27]show that malwares, such as worms, which are readily available online, can facilitate lateral movement across networks, allowing attackers to reach and compromise a PLC in the target network after initially compromising a PLC in a different network.

  - **Physical Attacks:** Physically connecting to a PLC offers the most reliable and powerful means of control. While direct access is unlikely for PLCs located within secure facilities (except for insiders), decentralized PLCs situated in publicly accessible areas due to their connection to remote sensors or actuators are more vulnerable.
  - **Social Engineering:** Attackers can manipulate employees to gain access to a PLC, using techniques like phishing for credentials or distributing malicious USB drives. These methods can introduce malware into the system, potentially reaching the target network.

- **UART Level Control:** To exploit vulnerabilities at the data-link layer, the ability to send and receive UART packets is essential. This can be trivial when an attacker gains full control of a PLC and can inject arbitrary code. However, it may be more challenging if the attacker's access is limited to sending application-level packets. In this work, we assume that the attacker achieves UART level control. However, future research could demonstrate that most of the attacks presented here can be adapted to function without this specific level of control.

4.4. Attack Surface

Industrial Control Systems present a complex and diverse attack surface due to their intricate engineering and interconnected components. This research focuses on the error management mechanisms within the protocol stack as the primary attack surface. The stack comprises the RS-485 standard at the physical layer, the UART protocol at the data-link layer, and PROFIbus at the application layer, with PROFIsafe added for safety-critical systems. Each layer exhibits behaviors that can be exploited as security vulnerabilities, enabling attackers to generate errors with significant impacts on the system.

RS-485 and UART:

- **Arbitration:** The lack of arbitration mechanisms in UART, unlike protocols such

as Ethernet and CAN, allows for unmanaged collisions. This can be exploited to perform stealthy fault injections, for example simulating electromagnetic interference. We experimentally demonstrated the dominance of '0' bits in RS-485, enabling an attacker to disrupt communication.

- **Flow Control:** UART's hardware and software flow control mechanisms, intended to prevent conflicts, are both software-controlled. An attacker can easily disable them or manipulate the Driver Enable (DE) pin in RS-485 to override ongoing transmissions.
- **Error Management:** UART lacks proactive error management, relying only on error detection through parity bits. The errors are expected to be analyzed at higher levels, and any action to protect the network is delegated to PROFIbus, but this leaves the system vulnerable to fault injection attacks.

PROFIbus:

- **Arbitration:** PROFIbus does not implement any additional arbitration mechanisms at the application level, relying on the Controller-Device architecture and cyclic communication inherited from the underlying layers. This design assumes that no collisions will occur, as only the Controller initiates communication and Devices respond sequentially. However, in an adversarial environment, this assumption is flawed and exposes the network to potential disruptions.
- **Error Management:**
  - **Network Blocking:** In the event of a Device failing to communicate with the Controller (i.e., not responding), the entire network enters a waiting state until the Device recovers. This prioritizes system safety but increases vulnerability to Denial of Service attacks.
  - **Dynamic Re-parameterization:** When a Device's watchdog timer expires, the Device assumes it is no longer controlled by the Controller and enters a state ready for re-parameterization. This creates a window for potential malicious reconfiguration. This behavior is essential for network initialization and Controller redundancy, allowing a redundant Controller to take control in case of the main Controller's malfunction.
- **Authentication:** Each message is processed by the PLC only if it originates from the expected device. Devices accept frames only from their Controller, while the Controller accepts frames only from the Device it last queried. However, this is not

true authentication, as attackers can spoof source addresses in the SA field, thereby potentially commanding the network.

- **Encryption:** The lack of encryption facilitates understanding network activity and crafting malicious frames. However, as discussed by Fauri et al. [11], introducing encryption in ICS may not always enhance the system, even from a security perspective.

PROFIsafe:

- **Monitoring Number Reset:** The Monitoring Number is a counter used in safety-related messages, and also serves a security function. It is not sent in clear text but is included in the calculation of the CRC, making it difficult (but possible, as proved by Akerberg et al. [30]) to find and manipulate. However, the MNR resets to 0 every time a Device enters the Passivation state (watchdog expires) and then resumes communication with the Controller. Since attackers can induce this error state in a target Device, it is also possible to know the Monitoring Number.
- **Inherited PROFIbus Vulnerabilities:** PROFIsafe is implemented on top of PROFIbus, and its vulnerabilities are not fully removed in PROFIsafe. Therefore, it inherits some of the problems of PROFIbus, impacting the overall security of the system.

4.5. Impact

This section analyzes the potential impact of exploiting the vulnerabilities presented in this threat model. As these vulnerabilities pertain to communication protocols, their impact can be far-reaching and affect a multitude of systems. The specific consequences of an attack will depend heavily on the targeted system and the nature of the threat agent carrying it out.

PROFIbus:

- **Denial of Service:** successful exploitation of PROFIbus vulnerabilities can result in a prolonged and disruptive DoS attack, effectively halting the entire targeted network. This attack is not self-recovering, necessitating manual intervention by an operator to power cycle the affected PLC. In scenarios where the targeted PLC is located in a hard-to-reach area, such as a hazardous environment, underground, or at high altitude, the resulting downtime can be extensive. The primary assets at

risk in this scenario are industrial production, leading to direct financial losses and potential reputational damage, and essential services, causing disruptions to large areas and populations.

- **Device Control:** By exploiting vulnerabilities, an attacker can seize control of any Device PLC within the same network as the compromised PLC. Targeting critical Devices, such as those controlling actuators, allows for dangerous and disruptive actions. This attack poses a threat to all identified assets, with the most severe consequence being the potential loss of human life. An attacker could manipulate dangerous robots, trigger explosions, and compromise safety systems, leading to catastrophic outcomes.

PROFIsafe:

- **Denial of Service:** PROFIsafe vulnerabilities can be exploited to initiate both targeted and widespread DoS attacks. While PROFIsafe's error management mechanisms offer some resilience compared to PROFIbus systems, the impact of a DoS attack in safety-critical environments remains extremely dangerous. A malicious actor could target PLCs responsible for safety systems, rendering them inoperable and transforming the industrial process into a life-threatening situation for personnel and potentially the public. For instance, an attacker could disable emergency shutdown systems in a chemical plant, leading to a potential disaster.
- **Device Control:** Similar to the PROFIbus scenario, an attacker gaining control of a Device PLC in a PROFIsafe environment can manipulate critical devices and safety systems. However, the consequences are magnified due to the inherent safety-critical nature of the systems involved. This type of attack represents one of the most significant threats in the cybersecurity landscape. For example, an attacker could override safety interlocks in a nuclear power plant, potentially leading to a meltdown with catastrophic consequences.

5 | Attacks

In this chapter, we will conduct an in-depth examination of the two attack paths that confirm the risks associated with the vulnerabilities identified in the threat model presented in this work. Initially, we will focus on the attack targeting PROFIbus, analyzing the steps and techniques employed. Afterwards, we will showcase a method for exploiting certain PROFIsafe vulnerabilities, thereby enhancing the efficacy of attacks previously proposed in research literature. Both of these attacks have the potential to result in Denial of Service and Device Control, the severity of which is contingent upon the expertise and knowledge possessed by the attacker.

5.1. Reparameterization Attack (PROFIbus)

The purpose of this attack is to reparameterize an arbitrarily chosen Device on the network.

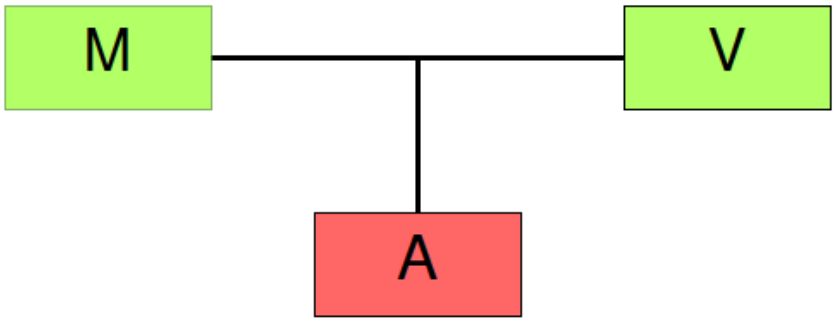


Figure 5.1: Attack scenario

Let us consider a scenario comprising a Controller (Master) (**M**), a victim Device (**V**), and an attacker Device (**A**). The presence of any additional Devices, regardless of their number, is inconsequential to this analysis. **A**'s objective is to re-parameterize **V**. To achieve this, **A** will initially disrupt the communication between **M** and **V**, causing **V**'s watchdog to expire. Subsequently, **V** will transmit a diagnostic frame and transition into the Parameterization state. At this point, **A** will preemptively send the parameterization

telegram before **M**, maliciously configuring new parameters for **V**. **V** will be isolated from the network, as it will reject any further frames from **M**, and will be susceptible to control exclusively by the attacker **A**.

This is the algorithm that we are going to explain in detail and the related flow diagram:

Algorithm 5.1 Reparameterization Attack

```
1: listenForControllerToTargetCommunication
2: while True do
3:     disturbCommunication
4:     if communicationBroke then
5:         break
6:     end if
7: end while
8: sendParameterization
9: sendConfiguration
```

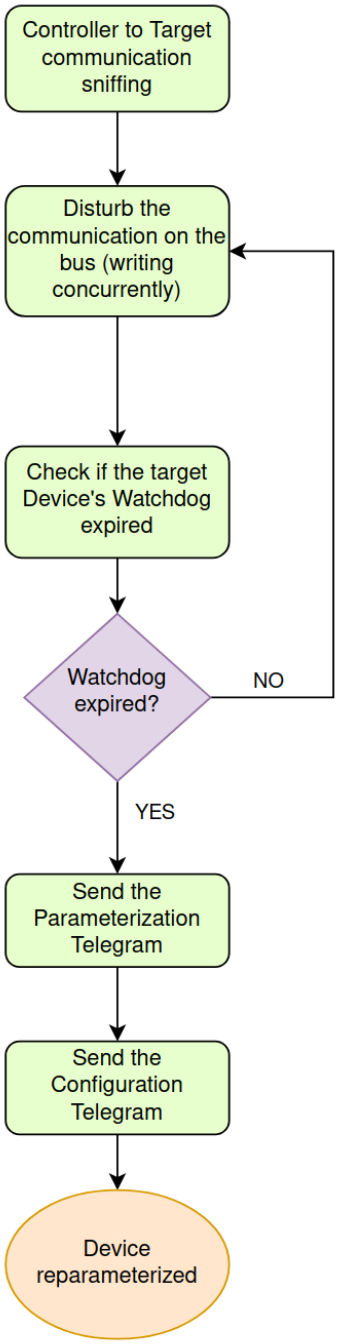


Figure 5.2: Reparameterization Attack Flow Diagram

Step 1: Controller to Target communication sniffing

The initial step for the attacker is to determine the precise moment to act. As PROFIbus frames are not encrypted, it is relatively simple to identify which device is transmitting and who the intended recipient is by merely monitoring the traffic on the bus.

In our assumptions, we posited that the attacker has the capability to read UART frames, such that when it detects the Controller communicating with the victim, it has sufficient time to corrupt that frame before it is fully written on the bus.

Reading and Overwriting the same frame When the Controller initiates writing on the bus, the Attacker can read five UART frames and verify that the value of the first is 0x68, while the fifth must be the address of the Device chosen as the target (Victim). The value of the first byte is checked simply as confirmation that we are reading the beginning of a frame. Now the attacker knows that this is the moment of communication between M and V and must swiftly corrupt this frame.

Anticipating the read The attacker may also encounter challenging time constraints, for instance, due to poor performance of the PLC or very high baud rates. In this scenario, it is preferable to anticipate the read. PROFIbus is a deterministic protocol, and in its cyclic communication, the Devices are always queried in the same order, with strict real-time constraints, to maintain a stable exchange time. This makes it very easy to predict with high accuracy when a specific Device is going to be queried. The attacker can monitor the bus, awaiting the communication between the Controller and the Device that is in the list just before the target, and it will be trivial to calculate the time to know when to write and disrupt the next communication.

Step 2: Disturb communication on the bus

The objective of the attacker in this step is to compromise the integrity of the frame that is currently being transmitted on the bus.

When the Victim receives a corrupted frame, it will not process it, so after a few failed attempts, the watchdog will expire. The Attacker can exploit the absence of arbitration and proper flow control to write on the bus concurrently with the Controller and generate enough noise to disrupt the frame.

The challenge of the attacker is to write some bytes on the bus to ensure that the frame is not received correctly. We do this analysis assuming that '0' is the easiest value to force on the bus (from experiments), but analogous statements can be made in the case of an environment in which '1' would be the easier value to force.

Writing one UART frame (8-bit payload) with value 0x00 is enough to ensure the frame is broken. This is both for reasons at the UART level and at the PROFIbus level:

- **UART level:** writing '0's overwrites the end bit of the Controller's frame, which

should be a ’1’. If the receiver does not read a ’1’ when it expects it at the end of the frame, a "Framing Error" is triggered, and will be fixed only when the next edge ’0’->’1’ is detected. At this point, the communication is fully desynchronized, and the received data are random and have no meaning anymore, but anyway, having detected an error, the receiver does not even process the data. This works in any case, except if the Attacker writes the frame at the exact same moment in which also the Controller writes its UART frame, because the end bit will not be overwritten. This case can be solved either by modifying the settings of the UART transceiver, if possible, so that the frame is longer, or can be addressed at the PROFIbus level.

- **PROFIbus level:** writing ’0’s imposes that one field of the received frame has value 0x00. This value is not allowed in several fields of the PROFIbus frame: Start Delimiter (SD), Data Length (LE/LEr), Destination Address (DA), Source Address (SA), End Delimiter(ED). In case the attacker reads and writes in the same frame time, only SA and ED are valuable targets; if instead, the attacker anticipates the read at the previous Device, all of them are valuable targets. This error causes the receiver to not process the frame at the application level.

SD2	LE	LEr	SD	DA	SA	FC	DSAP	SSAP	DU	FCS	ED
68H	X	x	68H	xx	xx	x	3CH	3EH	x..x	X	16H
				←----------VAR LENGTH----------→							

Figure 5.3: Fields that cannot have value ’0’

Step 3: Check Target’s Watchdog expiration

In this step, the attacker attempts to determine if the watchdog of the Victim has expired.

When a Device’s watchdog expires, it transmits a Diagnostic Telegram. The purpose is to inform the Controller of the problem, but in this context, it is very useful for the Attacker because it is proof that the exploit is working, and also this frame can be corrupted to delay the intervention of the Controller. The diagnostic telegram is easily detectable by monitoring the traffic since it uses the Function Code (FC) 0x0D.

The Controller does not retransmit its frame immediately but, according to the protocol principles, first continues the cyclic exchange with the other Devices. This means that the Attacker can keep listening on the bus until the Controller is going to communicate again with the target; in this case, we go back to the previous step. If instead, the Diagnostic

Telegram is detected, and possibly corrupted by the attacker, the exploit can proceed to the reparameterization steps.

Step 4: Send Parameterization Telegram

The Attacker sends a Parameterization Telegram to the Victim. This step is necessary because, otherwise, the Controller would perform the reparameterization at the first available cycle in a few milliseconds, and the Attacker would not be able to control the Device properly, thus resulting in a very reduced threat.

7	6	5	4	3	2	1	0
Lock_Req	Unlock_Req	Sync_Req	Freeze_Req	WD_On	*Reserved –Set to 0*		
WD_Fact_1							

Figure 5.4: Parameterization Telegram DU Byte 1

The parameters always present, by standard, in this frame are: Watchdog On (WD_On), Watchdog Time, Slave Reaction Time, Freeze/Sync Mode Enable, Lock/Unlock Slave, Group Assignment, Master Address, and Identification Number. Let us analyze the main targets for the Attacker:

- **Master Address:** sets the address that the Device will store as Controller, the only one from which it will accept frames. The Attacker can set an arbitrary number between 1 and 127 (possibly avoiding other Devices addresses), and the Victim will not answer the Controller's frames anymore. Now the Attacker is the only device able to control the Victim.
- **WD_On:** the Attacker has to set this parameter to Off ('0'). It is important so that, in any case, the Victim will not go again to the Parameterization state, so that the Controller cannot reparameterize it.
- **Lock Slave:** when a Device is locked, it will not respond to Parameterization Telegrams from any other Controller. The Attacker does not need to set this parameter because once the Victim is in the Data Exchange state, it will not accept any further Parameterization Telegrams. The only way to reparameterize the Victim is for the Attacker to send it a Diagnostic Frame. Only after this, a Parameterization Telegram can be accepted; therefore, this parameter is only useful in this specific scenario.

Step 5: Send Configuration Telegram

In the final step, the Attacker sends a Configuration Telegram. Despite its name, this frame's sole purpose is to verify that the Controller (in this case, the Attacker) and the Device (Victim) share the same hardware settings (e.g., Baud Rate).

This is a straightforward step: the Attacker simply constructs the frame with the appropriate data and sends it immediately after the Parameterization Telegram. It is however crucial because, without it, the Victim will not transition into the Data Exchange state and would remain susceptible to reparameterization by the actual Controller.

Note: most of the necessary data to craft the frame should be readily available by monitoring the traffic. In the event that some configurations are still missing, the Attacker can simply perform the first three steps of this attack and, by allowing the Controller to reparameterize the Victim, capture the Configuration Telegram to obtain all the missing data.

Results

At the end of the exploit, this is the situation:

- **Controller:** the Controller persists in its attempts to reparameterize the Victim, but without success. In this state, the Controller also continues its cyclic communication with the other Devices, but all operations are halted because, according to PROFIbus principles, the Controller only functions if all Devices on the network are operational. After an indeterminate number of attempts, the Controller will stop its efforts to fix the Device and will trigger an Alarm.
- **Victim:** the Victim is in the Data Exchange state, ready to be controlled, but will only accept frames from the new, malicious Master Address, thus exclusively from the Attacker. This situation can only be fixed through a manual power cycle performed by an operator.
- **Attacker:** the Attacker acts like a normal Device in relation to the Controller but also possesses exclusive control over the Victim.
- **Network:** every other Device is in Fail Safe mode in which they do not operate, waiting for the Victim to be fixed.

Advantages

To illustrate the advantages of this attack, let us compare it to two other powerful techniques found in research literature:

- **Pretend to be the Controller:** once an attacker gains control over a PLC, they can simply start sending frames over the network, choosing the Source Address as that of the Controller, so that every other Device will execute their commands. However, this approach is very unreliable and unstable:
  - Traffic: PROFIbus' cyclic communication keeps the bus very busy, and expecting to send several frames without a high number of collisions is unrealistic.
  - Controller still working: the real Controller is still controlling all the Devices, so the control from the Attacker will be partial and unpredictable.
  - No DoS: without reparameterizing a Device, the Controller will always be able to control all the Devices, so that, even if the attacker succeeds in generating a DoS, this will automatically be fixed by the Controller.
- **Change the Victim Address:** this technique consists of dynamically modifying the address of the Victim Device through the Set_Slave_Add Telegram. This can be done when the Device is in the Parameterization State, similarly to our attack, but the main problem is that this feature is disabled by default in all PLCs. Essentially, the effects of this attack and our attack are similar, but our attack is always feasible, unlike this one, and also the reparameterization of the Master Address grants the Attacker exclusive control over the Victim.

Other advantages pertain to the stealthiness of the attack. Anomaly-based Intrusion Detection Systems will have difficulty detecting it since the UART frames that the Attacker sends can be modified and partially randomized while still ensuring the disruption of the Controller's frame. In this way, the attack would be indistinguishable from any electromagnetic interference. Also, IDSs that check for Semantic Attacks would not easily detect it because all the state transitions are natural and again indistinguishable from the normal behavior of the system.

Reproducibility on PROFIsafe

PROFIsafe does not permit the dynamic modification of parameters, and naturally, in the event of a watchdog expiration, it would not transition to the Parameterization State. Consequently, this attack cannot be replicated on PROFIsafe with the same outcomes.

Nevertheless, the reaction to the expiration of the watchdog in PROFIsafe (referred to as F-Monitoring Time) remains a potential target for an attacker aiming to cause a Denial of Service. This can be particularly hazardous in safety-critical systems.

5.2. PROFIsafe CRC2 Manipulation

The objective of this attack is to discover the secret parameters used by a Controller-Device pair to generate the CRC2 in PROFIsafe. If an attacker can generate a valid CRC2 for a different pair, they gain control over the Device and can even impersonate the Device in communication with the Controller.

This attack concept was initially proposed by Åkerberg and Björkman in 2009 [30]. In this work, we present significant improvements to their technique and analyze the feasibility of the attack on the latest versions of PROFIsafe.

Background

The Cyclic Redundancy Check (CRC) in PROFIsafe is a vital mechanism for maintaining the integrity of safety-critical communication. It serves as a checksum to detect errors during message exchange between the Controller (safety host) and Device. PROFIsafe employs different CRCs (CRC1 and CRC2) to accommodate varying safety data lengths and processing requirements. CRC1 is calculated over initial safety parameters and forms the basis for CRC2, which is calculated cyclically for each data packet. The CRC2 calculation incorporates the CRC1 signature, safety I/O data, Status/Control byte, and Consecutive Number. The resulting CRC signatures are appended to safety messages, and the receiver verifies their correctness by performing the same computations. Any mismatch indicates an error in transmission. PROFIsafe supports two safety data lengths with corresponding CRCs to meet SIL3 safety integrity levels. The Virtual Consecutive Number (VCN), included in the CRC2 calculation, aids in detecting communication errors and monitoring delays.

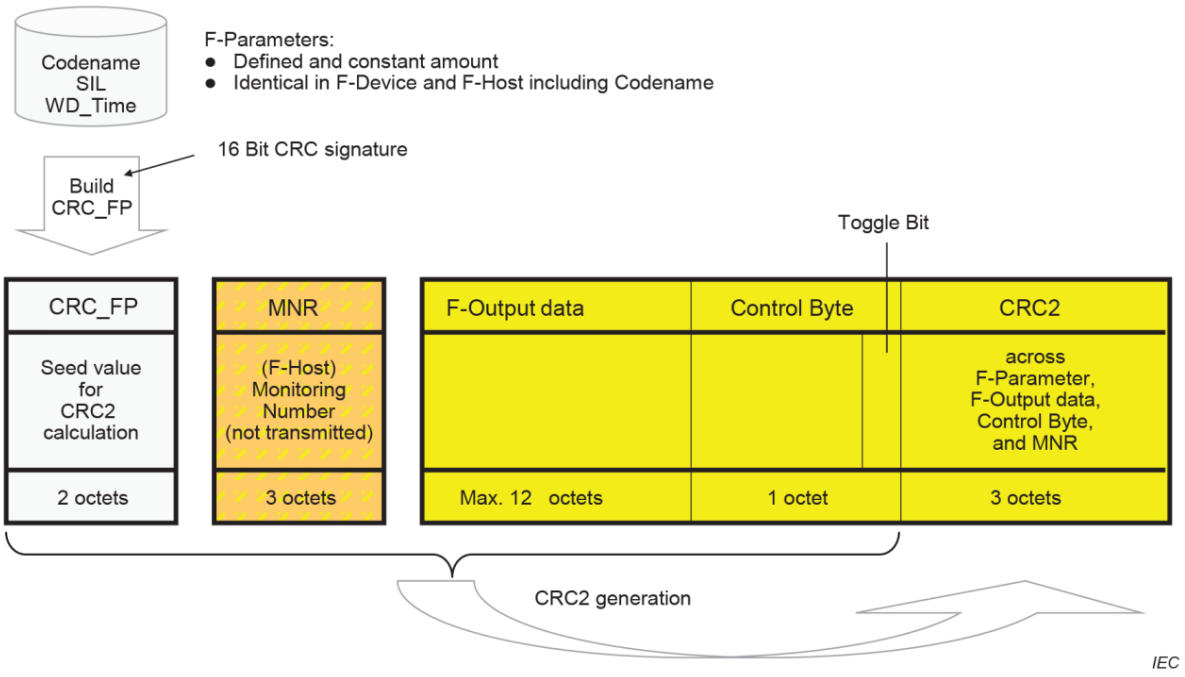


Figure 5.5: CRC2 Calculation

Åkerberg and Björkman's Idea

To successfully recalculate CRC2 and circumvent the data consistency check, the following information must be obtained:

- The current VCN (Virtual Consecutive Number)
- The codename for the sender and receiver
- The actual set of safety parameters

Acquiring this data is challenging because safety parameters are not transmitted after system startup, and the VCN and codename are always used in the CRC2 calculation, making them inaccessible to an attacker.

Åkerberg and Björkman observed that CRC1 is calculated using the sender and receiver codenames and the current safety parameters. Additionally, the standard specifies that CRC1 remains constant throughout the session. This knowledge significantly simplifies attempts to bypass the data consistency check. To launch a security attack, the following information is still needed:

- The current VCN
- The CRC1, which remains static throughout the session

By receiving one safety container and employing brute force to calculate all valid CRC1 and VCN combinations that produce the same CRC2 as in the received message, a set of potential CRC1 values can be obtained. Knowing that CRC1 is static during the session, the remaining combinations can be narrowed down to the actual CRC1 in use. This requires an iterative process that continues until the correct CRC1 is identified:

Algorithm 1. Retrieving the $CRC1$ and VCN

1: Receive a safety container
2: Calculate and store all permutations of VCN and $CRC1$ that generate the same $CRC2$ as the received safety container in step 1
3: **repeat**
4: Receive another safety container
5: **for all** stored permutations of $CRC1$ and VCN from step 2 **do**
6: Calculate $CRC2$ of safety container from step 4 with stored permutation of $CRC1$ and VCN
7: **if** calculated $CRC2$ = received $CRC2$ and $VCN <$ previous VCN **then**
8: Discard the permutation as a possible candidate
9: **end if**
10: **end for**
11: **until** one or zero valid permutations of $CRC1$ and VCN remain

Figure 5.6: Akerberg's Algorithm

Once the CRC1 is discovered, the attacker only needs to find the current VCN. This is achieved by attempting to recalculate CRC2, starting from the last known VCN and incrementing by 1 until the current VCN is found.

Test

Åkerberg and Björkman tested their algorithm on a 2 GHz laptop. The brute-force process to find all CRC1-VCN combinations took 11 hours, and selecting the correct CRC1 took 2 minutes. Finding the VCN also required a negligible amount of time. Clearly, the main bottleneck of the technique lies in the first step: 11 hours is a significant duration, and the laptop used for testing might be hundreds of times more powerful than some PLCs. Only very high-end PLCs could likely execute the algorithm in a comparable timeframe, albeit still longer, while for many other PLCs, obtaining the CRC1 might take weeks or even months, assuming it remains unchanged during that period. In the next section, we will explain how to drastically reduce this time.

New Idea: Reset the Virtual Consecutive Number

Through in-depth examination of PROFIsafe's Error Management mechanisms, we discovered that in the event of a severe communication fault, such as the expiration of the F-Monitoring Time, both the Controller and Device reset their VCN to 0.

The attacker can simply trigger the F-Monitoring Time expiration, as described in the previous attack on PROFIsafe, and will automatically know the VCN of the next exchanged frame. This significantly simplifies Åkerberg and Björkman's algorithm, as one of the two unknowns is known without any computation. The following diagram shows graphically the new algorithm:

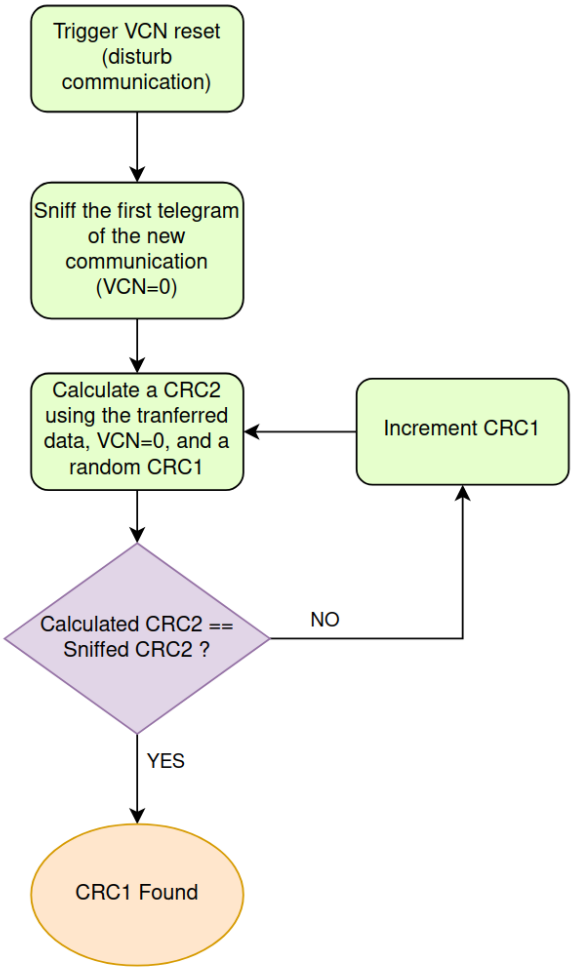


Figure 5.7: New CRC Manipulation in PROFIsafe

Speed-Up

The original algorithm required $2^{16} * 2^{24} = 2^{40}$ CRC2 calculations (CRC1 * VCN). With

the knowledge of the VCN, only the possible number of CRC1s needs to be explored, which is 2^{16}. This results in a speed-up factor of $2^{40}/2^{16} = 2^{24} = 16,777,216$.

In the test setting of Åkerberg and Björkman, the entire calculation would now take approximately 2.5 ms. This significant improvement makes it feasible for even low-end PLCs to manipulate the CRC2.

Latest PROFIsafe version and F_CRC_Seed=1

The most recent version of PROFIsafe, 2.6.1 (IEC-61784-3-3:2021), was released in 2021 and introduces a new method for managing the VCN and calculating the CRC. This option, enabled by setting the parameter F_CRC_Seed=1 in the GSD (configuration) file, is not yet widely adopted in most PLCs. For example, Siemens supports it in the latest releases of the S7-1200 and S7-1500, which are their most expensive and high-end PLCs, but not in other models. It is worthwhile to analyze how to bypass this new CRC management, as it once again highlights the lack of proper security considerations from the protocol designers, even in 2021.

The two relevant changes for our purposes are:

- **VCN:** It still resets in case of faults, but to a value called CRC_FP+, which is a 32-bit CRC similar to CRC1, calculated based on safety parameters. The VCN increment is no longer by 1 but depends on a secret variable called Modifier and the Codename parameter.
- **CRC2:** It is fixed at 32 bits and calculated over packet data, VCN, and the Control Byte. The algorithm is based on the previous one.

This table summarizes the changes in Virtual Consecutive Number management:

MACRO	ACTION (Pseudocode)
INITIALxH	old_MNR = CRC_FP++; **MNR=CRC_FP++;** **CRC2 = CRC2 XOR Modifier;** CN_incrNR_64[1] = 0x5851F42D4C957F2D x Codename; SwapHL=shl(CN_incrNR_64 [1], 32) + shr(CN_incrNR_64 [1], 32); CN_incrNR_64[2]=0xAB16D2792302FE5A x (SwapHL + Modifier) + 1;
RESETxH	R_cons_nr = 1; **MNR=CRC_FP++;** **CRC2 = CRC2 XOR Modifier;** CN_incrNR_64[1] = 0x5851F42D4C957F2D x Codename SwapHL=shl(CN_incrNR_64 [1], 32) + shr(CN_incrNR_64 [1], 32); CN_incrNR_64[2]=0xAB16D2792302FE5A x (SwapHL + Modifier) + 1;
RUNxH	R_cons_nr = 0; old_MNR = MNR; CN_incrNR_64 [0] = CN_incrNR_64 [1] + CN_incrNR_64 [2]; CN_incrNR_64 [2] = CN_incrNR_64 [1]; CN_incrNR_64 [1] = CN_incrNR_64 [0]; **MNR = Most significant 32 Bit of CN_incrNR_64 [0];**

Table 5.1: New VCN Management

The new way of calculating and incrementing the VCN is an improvement from a security perspective. The attacker now faces three unknown variables: Modifier, Codename, and CRC_FP+. This poses significant challenges to Åkerberg and Björkman's technique, as the VCN changes unpredictably. However, using the new idea introduced in this section, we can still manipulate the CRC.

Crack the new PROFIsafe

Current version

The primary issue with this PROFIsafe release is that the Modifier is merely a variable intended for future use and currently always set to '0': "Modifier is an F-Host generated value for future use, which determines a new initialization. However, within this release,

the value ’0’ is always used within the F-Host-macros INITIALxH and RESETxH." [9].

This reduces the unknowns to just two variables, both of which remain static throughout the entire session (at least). The attacker can crack the CRC mechanisms in a few steps:

1. Trigger a fault, causing the VCN to reset to CRC_FP+.
2. Read the Controller -> Device frame, recalculate the CRC2 with every possible value of CRC_FP+ (2^{32}) and identify the used CRC_FP+ value.
3. Read the Device -> Controller frame, try every possible value of Codename (2^{16}, as the Controller address is known) and determine the Codename. (Optionally, narrow down the solution set using subsequent frames)
4. Calculate each subsequent VCN and manipulate the CRC using the knowledge of CRC_FP+ and Codename.

The main step of this attack involves 2^{32} CRC2 calculations, which is 256 times faster than Åkerberg and Björkman’s technique on the older PROFIsafe version.

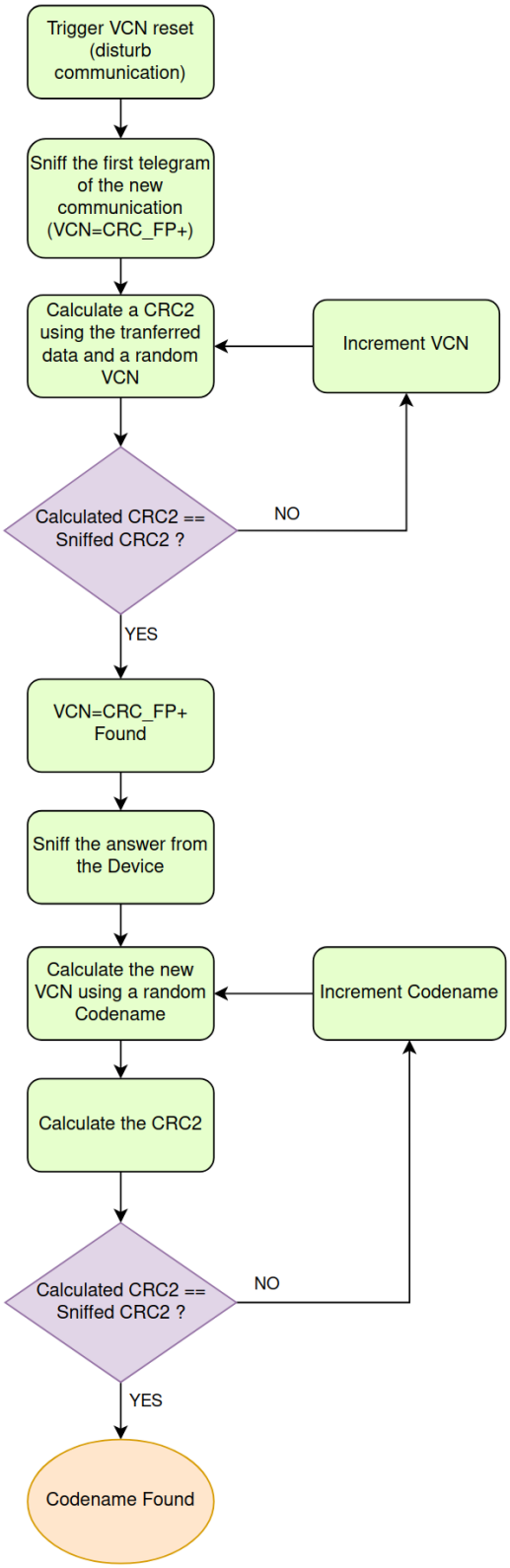


Figure 5.8: New CRC Manipulation in PROFIsafe from version 2.6.1

Future version

Not utilizing the Modifier variable is a significant oversight, considering the time required to release a new PROFIsafe version and implement it in new PLCs. This leaves most existing PLCs vulnerable to the attack described in the previous section. However, it's interesting to see how the situation changes for the attacker if the Modifier variable is used correctly.

Following the same ideas presented thus far, the attacker can:

1. Trigger a fault, causing the VCN to reset to CRC_FP+.
2. Read the Controller -> Device frame, recalculate the CRC2 with every possible value of CRC_FP+ (2^{32}) and perform an XOR with the transmitted CRC2. This yields 2^{32} pairs of (CRC_FP+, Modifier).
3. Read the Device -> Controller frame, try every possible Codename value for each (CRC_FP+, Modifier) pair found previously, resulting in all feasible triplets (CRC_FP+, Modifier, Codename). There should still be around 2^{32} such triplets.
4. At the next Controller-Device frame, test all triplets to see if they produce the same CRC2, ultimately finding the used set of values.

We can observe a significant increase in calculation time for the attacker. Due to Step 3, the algorithm has a complexity of $O(2^{48})$, 256 times more than the original Åkerberg and Björkman attack. This might seem positive, as it translates to roughly 4 months for a 2 GHz laptop and even longer for a PLC. However, it's not sufficient.

The encryption technique remains too weak to be considered secure. For instance:

- **Data Exfiltration:** If the attacker can send the data from the two frames back to themselves from the PLC (not difficult, as it's a common attack), the brute-forcing time would drastically decrease.
- **Algorithm Enhancements:** It's highly likely that, with deeper analysis of real-world scenarios, the algorithm can be enhanced. For example, CRC_FP+ is calculated based on the F parameters, which include the Codename. Since the attacker is on the same network as the target, several parameters are the same and already known to the attacker. They could generate all possible CRC_FP+ values starting from the Codenames (and other parameters), obtaining (CRC_FP+, Codename) pairs much faster and drastically reducing overall calculation times. The formal analysis and definition of this technique are topics for future work.

In summary, PROFIsafe version 2.6.1 is still vulnerable to the attack that combines Åkerberg and Björkman's technique with our new idea regarding the Error Management

mechanisms. It lays the foundation for better CRC management, but even with the increased difficulty for the attacker, it doesn't yet solve the CRC Manipulation problem.

6 | Countermeasures

This chapter proposes defense mechanisms to prevent the successful execution of the attacks explored in the previous chapter. Our focus is on addressing underlying vulnerabilities in systems, rather than providing specific exploit mitigations. This approach aims to establish a foundation for more secure Industrial Control Systems.

Popular and simple current defense mechanisms, as outlined in the State of the Art chapter, are insufficient to mitigate the attacks presented here. Consider the Reparameterization Attack against PROFIbus:

- **Anomaly-based detection:** While the attack initiates with Fault Injection, the inherent flexibility and non-deterministic nature of the fault make it extremely challenging for anomaly-based IDS to differentiate from legitimate faults (e.g., Electromagnetic Interference).
- **Semantic approach detection:** The attack leverages normal protocol operations, without unusual transitions or alterations in frame periodicity. This makes it likely to evade detection by IDS employing semantic approaches, such as the one proposed by Li et al. [17].
- **Encryption:** For decades, researchers have attempted to implement encryption in Industrial Control Systems to enhance security, particularly at the data-link level, thus preventing attacks like those discussed in this work. However, the real-time nature and specific characteristics of these systems make the use of encryption techniques impractical. Watson et al. [28] elaborate on the challenges of applying traditional encryption algorithms to PROFIbus, and highlight that currently feasible algorithms are easily compromised.

Regarding the PROFIsafe attack, the previous chapter demonstrated the inadequacy of existing countermeasures implemented in the latest release.

This table summarizes the defense mechanisms we propose and explain in the following sections:

Target	Countermeasures
UART	- Arbitration (collision management)
PROFIbus	- Static parameters - Intrusion Detection System (monitor parameter modifications)
PROFIsafe	- Frequent random keys exchange
System	- Intrusion Tolerance (best practices to minimize damage)

Table 6.1: Countermeasures

6.1. Protocol Countermeasures

This section explores potential defense mechanisms for each layer of the stack. The goal of these countermeasures is to enhance the security of ICS environments and improve the long-term security of the protocols themselves. However, these mechanisms can only be implemented by the designers of the protocols. They require considerable time for proper development, testing, and deployment. Thus, they are not suitable for immediate attack prevention, as they cannot be implemented by final application developers.

Arbitration (UART/RS-485)

The attacks described in this work originate at the data-link level. The first vulnerabilities exploited are the lack of arbitration in both the standard RS-485 and the UART protocol. Arbitration would provide collision management and recognition, significantly impeding an attacker's ability to disrupt the Controller's frames without detection. Such mechanisms can be modeled after existing arbitration methods in other data-link layer protocols like CAN and Ethernet.

While implementing arbitration at the data-link level does not guarantee complete system security, as these mechanisms can sometimes be abused or may leave other possibilities open to attackers, it represents a fundamental defense mechanism that would substantially reduce the attack surface.

Static parameters (PROFIbus)

The primary vulnerabilities exploited in PROFIbus are the lack of authentication and the ability to dynamically reparameterize a Device. Authentication concerns are addressed by

PROFIsafe. Thus, for critical applications that cannot tolerate unauthorized actions under attack, the PROFIsafe protocol should always be used in conjunction with PROFIbus.

A straightforward solution to prevent reparameterization is to make the parameters static. The error mechanisms associated with reparameterization remain unchanged, however, if the Device detects a discrepancy between the received parameters, at least the most critical ones, and those initially provided at startup, an alarm must be triggered. This simple and effective solution protects the system against the attack we developed, as well as any other attack that might exploit the reparameterization vulnerability in PROFIbus Error Management.

Although this countermeasure conflicts with some features and principles of the protocol, we believe the security benefits far outweigh the losses:

- **Controller redundancy:** The reparameterization behavior is essential for automatically switching to a backup Controller in case of malfunctions. This can still be achieved by requiring the substitute Controller to have the same address as the main one. In this way, the alarm would not be triggered as the parameters remain unchanged, and there would be no network conflicts because the substitute Controller only starts communicating when the main one has stopped.
- **Dynamic parameters modification:** The ability to modify parameters while the system is running is lost. However, this feature should be used with extreme caution by developers, and in a well-designed system, it should never be necessary. Therefore, the countermeasure does not truly remove a fundamental feature of the protocol. Even PROFIsafe does not have this feature. When a parameter change is necessary, which should be infrequent in such environments, the entire system can be power cycled, and the configuration files (GSD) modified. Moreover, the only parameter that might require frequent modification during network setup is the Watchdog time. This parameter can be excluded from the static parameters affected by the countermeasure.

Frequent random keys exchange (PROFIsafe)

Considering the latest version of PROFIsafe (2.6.1) with the proper utilization of the variable Modifier, as explained in the "Attacks" chapter, this method for managing CRC and authentication still presents two security issues:

- **CRC2 Partially Based on Fixed Data:** The CRC2, appended at the end of the frame, serves as proof of authentication from the sender and a means to verify

message integrity. The only secret parameter in its calculation is the Virtual Consecutive Number (VCN). The calculation of this variable relies on safety parameters (CRC_FP+), Codename, and Modifier. Of these three, only the Modifier changes frequently, while the other two remain constant throughout the sessions. This provides an attacker with ample time to brute-force the two fixed data elements, after which forcing the Modifier value becomes trivial. Particularly in scenarios where the attacker can exfiltrate data and perform brute-force attacks in a faster, external environment, this vulnerability can be fatal.

- **CRC_FP+ Based on Safety Parameters:** The CRC signature used in the VCN calculation, CRC_FP+, is based on safety parameters. We have already discussed the problem of having data that does not change over time in this type of management. However, in this specific case, there is an additional concern. Most, if not all, safety parameters are related to and calculated based on network characteristics rather than PLC-specific factors. This means that when an attacker compromises a PLC, they already possess knowledge of almost all the parameters for any other target they may choose, making the calculation of another CRC_FP+ significantly easier and less randomized than intended.

Both vulnerabilities can be addressed simultaneously. Since the processing overhead of a modern encryption system is impractical, the solution lies in the frequency of key exchange. As previously analyzed by Rezai et al. [24], the overhead for key distribution is negligible at high baud rates, allowing this process to occur quite frequently, which is crucial to prevent an attacker from forcing a key that is still in use in real-time. Also, the CRC2 (or VCN) calculation should be based solely on random values generated by the Controller and transmitted to the Devices over the network.

The choice between maintaining the system with three secret variables (all random) or transitioning to a single longer key can be determined through testing on modern PLCs to identify the optimal solution within real-time constraints. Based on these findings, the key refresh interval can be established.

6.2. Software Countermeasures

This section explores techniques that final developers can employ to prevent or mitigate the attacks described. These solutions may be attack-specific rather than general protocol improvements, as they are intended for application developers who cannot modify the protocol implementation and may face delays in receiving the necessary security updates.

Intrusion Detection System: Parameters Modification Detection (PROFIbus)

As noted in the chapter introduction, general Intrusion Detection Systems might struggle to detect the attacks presented in this work. However, an attack-specific solution can be straightforward and lightweight to implement. This technique is based on the same concept as the "Static Parameters" countermeasure explained earlier. The key point is that if we prevent an attacker from reparameterizing a Device, the potential damage is significantly reduced.

All parameters are "centralized" as they are stored in GSD files and then transferred from the Controller to each Device at startup. The IDS can automatically check these files and, when reparameterization occurs, verify that sensitive parameters (e.g., Master Address) transmitted to that Device have not been modified. While this technique sacrifices some protocol dynamism, this trade-off has already been discussed in the previous section.

Intrusion Tolerance

Intrusion tolerance is a set of best practices and techniques, rather than a specific defense, aimed at reducing damage in safety-critical environments. Unfortunately, developers have limited options to address the issues highlighted at the PROFIsafe level without compromising real-time requirements. Therefore, it is essential to implement strict, precise, and secure rules to ensure that all actuators operate within predefined limits. While this does not guarantee absolute safety or security, as some components might still act dangerously even when operating close to standard behavior (e.g., precision manufacturing machines) or financial losses may still occur, intrusion tolerance remains crucial in preventing disasters to the greatest extent possible.

7 | Implementation

Following the analysis of protocol vulnerabilities and the development of potential attacks, we aim to construct a simulated environment that closely replicates real-world conditions. This involves recreating a system analogous to a fieldbus network deployed within an industrial control system (ICS).

7.1. Software Implementation

The software components required for our environment consist of implementations of the protocols at each level, excluding the physical layer. While open-source implementations of UART are readily available, the challenge arises with PROFIbus and PROFIsafe. The specifications of these protocols are open standards. However, their complexity, coupled with the fact that their real-world versions were primarily developed by Siemens, has resulted in the widespread adoption of the proprietary Siemens implementation among PLC producers. This proprietary nature, coupled with the high cost associated with industrial-scale applications, particularly for PROFIsafe, presents a significant obstacle.

Given these constraints, the most suitable solution identified is an open-source library called *PyProfibus*. This library provides a reliable implementation of the Controller PROFIbus stack and was developed by an engineer with expertise in automation and industrial development [8].

PyProfibus

PyProfibus is an Open Source PROFIbus-DP stack written in Python. It is able to run on any machine that supports Python, including embedded machines such as the Raspberry Pi or even tiny microcontrollers such as the ESP32 (Micropython).

Architecture

The framework is structured into three primary components:

- *phy*: This component includes modules and classes for interacting with various

physical interfaces, such as FPGA and Serial (RS-485). It also includes UART code and configuration.

- *fdl*: Fieldbus Data-Link layer, this middleware layer serves to decouple the lower-level UART and physical layers from the application layer (PROFIbus). Specifically, it provides an initial mapping of the raw data received into an intermediate frame format.
- *dp*: Decentralized Peripherals, this is the implementation of the PROFIbus standard (Controller only), in its slightly simplified DP version, which is the most widely adopted globally.

Each of these components also incorporates code for managing the *Transceiver*, a crucial element for reading and writing on the bus.

Testing

The author validated the reliability of *PyProfibus* by testing a network in which a Raspberry Pi 2 functioned as the PROFIbus Controller, running PyProfibus, and a Siemens ET200S (a widely used industrial PLC) and a Siemens S7-315-2DP served as Devices. This configuration resulted in a successful and operational fieldbus network.

These tests demonstrate the reliability and practicality of *PyProfibus* for testing the developed attacks within an environment that closely approximates a real-world ICS network.

PyProfisafe

Due to the absence of certain features necessary for our experiments within the *PyProfibus* library, we developed an extension, resulting in a new library called *PyProfisafe* [7]. The modules lacking in PyProfibus are the stack for the PROFIbus Device and the complete PROFIsafe stack.

PROFIbus Device Stack

The *phy* and *fdl* components of *PyProfibus* offer a robust and tested layer for PROFIbus communication. Thus, the only module requiring addition is at the *dp* (Application) level. This component was implemented using a State pattern to associate the Device object with its specific behaviors at different stages of communication.

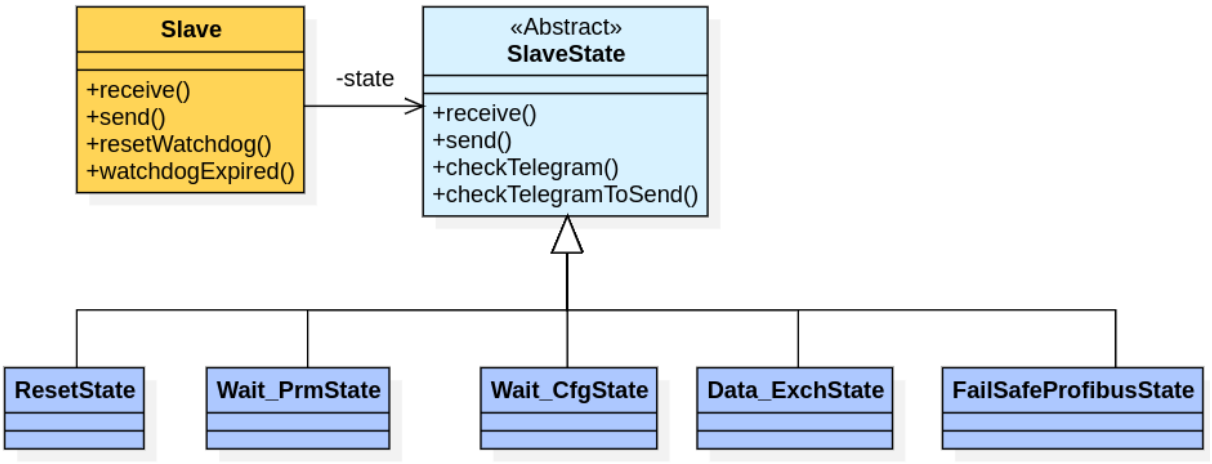


Figure 7.1: PyProfisafe state diagram

Furthermore, code was added to the Controller stack to fully support, in accordance with the standard, the Clear Mode (failsafe), which was not comprehensively addressed in *PyProfibus*.

PROFIsafe Stack

The PROFIsafe stack comprises the underlying PROFIbus stack, augmented by an additional layer responsible for incorporating PROFIsafe safety features. The two primary tasks of this layer are:

- **Frame encapsulation**: The PROFIbus frames are wrapped within PROFIsafe frames. This process necessitates the implementation of logic for the "Control Byte" and the calculation of the CRC.
- **Safety Behavior**: We introduced additional parameters, such as the F-Monitoring time. The system's response to potentially unsafe conditions (errors) is modified under this protocol. We implemented most of this functionality by extending the PROFIbus Controller classes.

Note: This PROFIsafe implementation is not yet fully comprehensive. For example, the precise calculation of CRC remains to be implemented, as developing a complete library of this nature is a highly complex and time-consuming effort that falls outside the primary scope of this work. However, all implemented features adhere to the standard and contribute to testing the developed exploits.

7.2. Hardware Implementation

Having selected PyProfibus as the software library, the next task was choosing suitable hardware. The required network setup consisted of three devices (one Controller, two Devices), transceivers, a bus, and a logic analyzer. The chosen components were Raspberry Pi Zero 2W devices, MAX485 transceivers, jumper wires, and a Saleae logic analyzer.

- **Raspberry Pi Zero 2W:** A compact single-board computer ideal for embedded applications. It features a quad-core 64-bit ARM Cortex-A53 processor at 1GHz, with 512MB of LPDDR2 SDRAM. Connects to the transceiver via pins 4-6-8-10 (5V, GND, UART TX, UART RX).
- **MAX485:** A widely-used integrated circuit functioning as an RS-485 transceiver. It converts digital signals from microcontrollers or devices like the Raspberry Pi into differential signals suitable for RS-485 transmission. It also features automatic Driver Enable (DE) and 120ohms resistors for realizing the RS-485 with jump wires.
- **Saleae Logic 8 USB Logic Analyzer:** A device for debugging and analyzing digital signals. It captures signals between Controller and Devices, visualizes waveforms, and decodes protocol frames. It has 8 input channels, each capturing up to 100 MHz signals with 2.5 ns resolution.

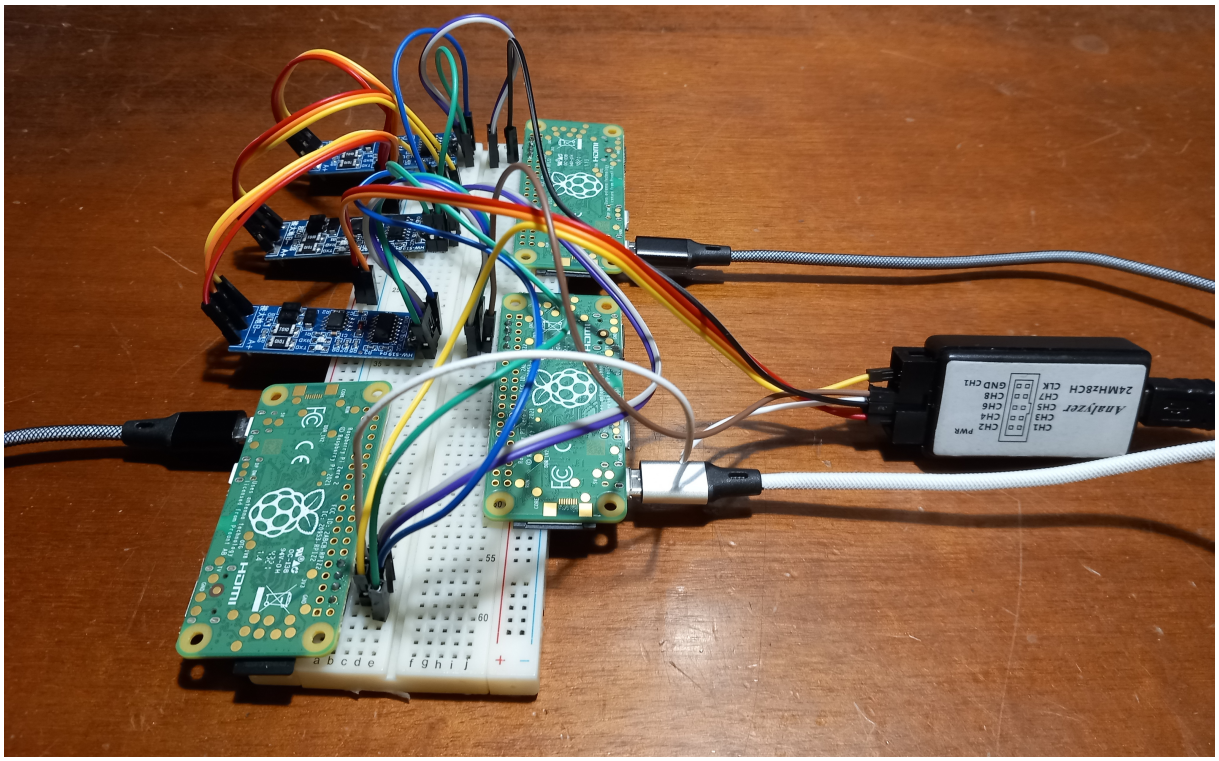


Figure 7.2: Hardware network

8 | Experimental Validation

This chapter presents the implementation and experimental testing of the concepts and techniques discussed theoretically in previous chapters. We focus on the attack against the PROFIbus protocol, demonstrating how vulnerabilities at each level of the stack can be exploited in a real-world scenario.

8.1. Reparameterization Attack (PROFIbus)

The initial testing phase ensured the attack was executed in a fully functional environment to eliminate the possibility of unknown factors influencing the results. We began with standard cyclic communication between a Controller and two Devices. One Device, designated as the Victim, ran an instance of the Device class. The other Device, the Attacker, ran both the Device class and the Exploit code, that is executed in the second phase of the test.

Hardware Configuration

- UART: BAUD_RATE = 9600bps; PARITY = NO; NUM_STOP_BITS = 1;
- INPUT_SIZE = 2 bytes (each channel)
- OUTPUT_SIZE = 2 bytes (each channel)
- Device_Victim: Add = 0; Master_Add = 111; Watchdog_ms = 4000; Group = 0;
- Device_Attacker: Add = 1; Master_Add = 111; Watchdog_ms = 30000; Group = 0;

Test

The mock application is straightforward. The Controller initiates communication by sending data exchange telegrams with a two-byte Data Unit (0x00, 0x00) to each Device. Each Device responds with the Data Unit incremented by 1 in each byte, and this process continues. The Controller verifies that the received message's Data Unit is always correctly incremented by 1.

In the test detailed in this section, we chose to launch the attack after 3 cycles. This demonstrates the attacker's ability to mimic a normal Device and initiate the attack from a standard data exchange scenario at will.

```
def testAttack(self):

    ### normal data exchange with master
    out_du = bytearray()

    for i in range(3):
        r = self.slave.receive(15)
        if not r:
            raise SlaveException("Did't receive anything!")
        print("Slave " + self.slave.getId() + " received the telegram: %s" % self.slave.rx_telegram)
        #answer the master sending back the same data incremented by 1
        in_du = self.slave.rx_telegram.getDU()
        if i == 0:
            out_du.append(in_du[0] + 1)
            out_du.append(in_du[1] + 1)
        else:
            out_du[0] = in_du[0] + 1
            out_du[1] = in_du[1] + 1
        send_telegram = DpTelegram_DataExchange_Con(
             self.slave.getMasterAddress(),
             self.slave.address,
             FdlTelegram.FC_OK,
             out_du
        )
        self.slave.send(send_telegram)

    ### Run the exploit
    self.attacker.run()
```

Figure 8.1: Attacker test code

At this point the attacker runs the exploit:

Algorithm 8.1 Reparameterization Attack

```
listenForControllerToTargetCommunication
while True do
    disturbCommunication
    if communicationBroke then
        break
    end if
end while
sendParameterization
sendConfiguration
```

Step 1: Controller to Target communication sniffing

This step proceeds as described in the "Attacks" chapter. The Attacker reads 5 bytes and verifies if the Controller is transmitting a frame to the Victim. A *deque* serves as a buffer, and the check is performed each time an item is added to and removed from the queue.

Step 2: Disturb communication on the bus

This is a crucial stage in our experiment, demonstrating the exploitability of vulnerabilities in the RS-485 and UART protocols. Our tests involved various methods to override the frame on the bus, and the results indicated that the easiest value to force is '0'.

In this capture we can see the moment in which the attacker overwrites the frame on the bus, transmitting a *bytearray* of 2 bytes with value 0x00. Considering that is transmitted in UART protocol the entire frames will follow the rules [Start Bit | Payload | End Bit]: 0|0000.0000|1|0|0000.0000|1

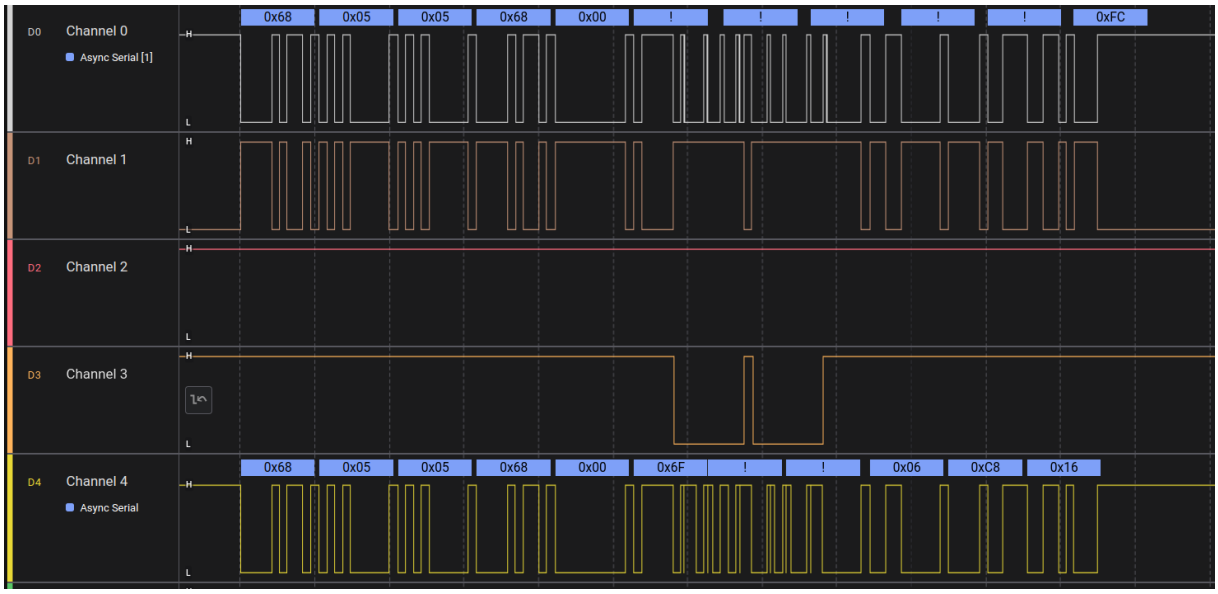


Figure 8.2: Capture: Attacker overwrites the Controller's telegram

In this setup, Channel 0 and Channel 1 represent the current bus activity. Using RS-485 notation, Channel 0 is the A+ channel, and Channel 1 is the B- channel. Channel 2 shows the Device Victim's transmission, Channel 3 the Device Attacker's, and Channel 4 the Controller's.

When the Attacker and Controller transmit simultaneously, the bus loses coherency. In

a functioning RS-485 system, channels A and B always have opposite values. However, in this scenario, they are essentially independent and incoherent. Channel B appears to follow the Attacker's activity with reversed signs, while Channel A reacts to both transmitters without properly adhering to either.

The next capture presents a similar situation from the receiver's perspective. Despite these inconsistencies on the bus, the Victim in Channel 2 accurately reads the two frames with a '0' payload sent by the Attacker. This triggers the Framing Error discussed in the theoretical approach.

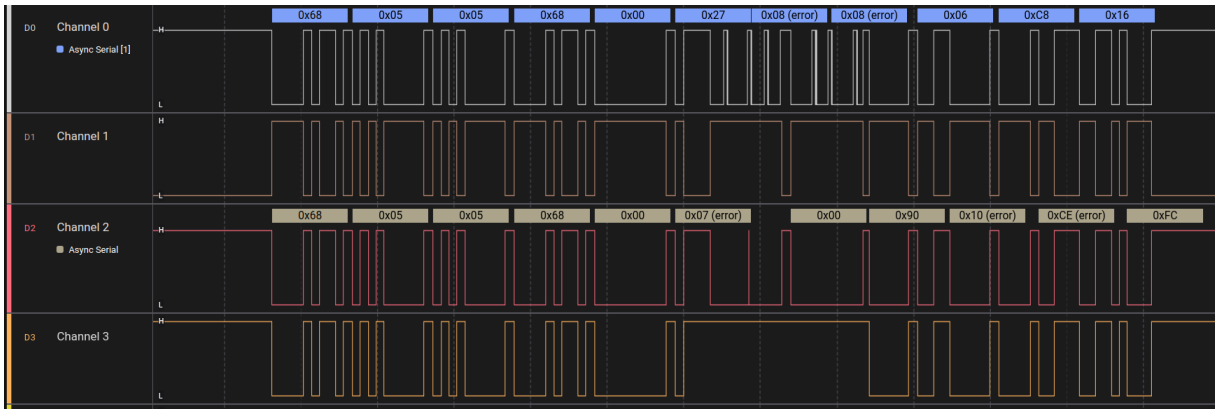


Figure 8.3: Capture: Victim receives the corrupted telegram

When this corrupted frame is received, the Victim outputs a log "Checksum mismatch", indicating an error at the physical level.

```
XXX| Slave victim in state <class 'pyprofibus.slave.Data_ExchState.Data_ExchState'> sends frame of type [<class 'pyprofibu
s.dp.dp.DpTelegram_DataExchange_Con'>] at time: 1722344728
XXX| DpTelegram(da=0x6F, sa=0x01, fc=0x00, dsap=None, ssap=None, du=05 05)
Checksum mismatch
```

Figure 8.4: Victim's log

Step 3: Check Target's Watchdog expiration

This step is handled similarly to Step 1, employing a *deque* and checks on the first and seventh bytes. In the capture, we observe the Diagnostic Telegram transmitted by the Victim. This frame is identifiable by the seventh byte with a value of 0x0D, representing the Function Code field of the PROFIbus frame.

Step 4: Send Parameterization Telegram

The Attacker transmits a malicious parameterization frame to gain control of the Victim. The code snippet below illustrates the injection of data into the parameter object. The first and last lines are noteworthy, showcasing the manipulation of the source address and the setting of the Status Byte, disabling the watchdog and locking the Victim.

```
self.prm_telegram = DpTelegram_SetPrm_Req(
    da=self.target_address,
    sa=self.fake_master_address,
    fc=0x4D,
    dsap=0x3D,
    ssap=0x3E
)
self.prm_telegram.wdFact1 = 0x00
self.prm_telegram.wdFact2 = 0x00
self.prm_telegram.minTSDR = 0x00
self.prm_telegram.identNumber = 0x00
self.prm_telegram.groupIdent = 0x02
self.prm_telegram.addUserPrmData(b'\x58')
self.prm_telegram.stationStatus = DpTelegram_SetPrm_Req.STA_LOCK # wd off
```

Figure 8.5: Parameterization Telegram Code

If the frame is successfully received and accepted, the Victim will promptly respond with an acknowledgment, as depicted in the capture below.

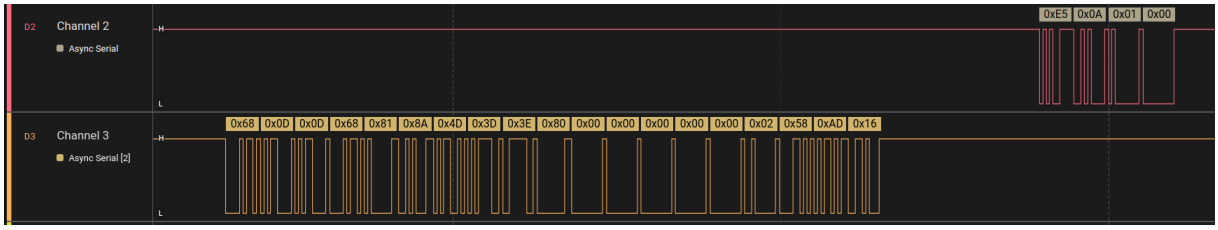


Figure 8.6: The Attacker reparameterize the Victim

Step 5: Send Configuration Telegram

This step is analogous to Step 4.

Limitations

The specific setup and tests conducted in this study have a few limitations that can be addressed and investigated in future research:

- **Environment:** While our experimental environment closely resembled real-world scenarios and adhered to the standard, we could not test on a fully third-party PROFIbus network due to the reasons outlined in the Implementation chapter. We do not foresee any characteristics or details that would prevent the attack or yield different results. However, we anticipate testing the exploit in entirely third-party environments in future work.
- **Baud Rate:** The technique of reading and overwriting the same frame, as implemented in our experiments, functioned correctly up to a baud rate of 93,750 bps. At higher speeds, the Controller had already completed its frame transmission by the time the Attacker initiated its write operation. This issue was addressed theoretically in the "Attacks" chapter using the "Anticipating the Read" technique. Future work will provide experimental validation of this technique.

9 | Future Works

This chapter first outlines the limitations of this thesis and proposes how future work can address them. Subsequently, we will highlight the most interesting findings and explore potential avenues for future research in those directions.

9.1. Limitations

The primary limitation encountered in this research pertains to testing difficulties. We could not test our Reparameterization Attack on PROFIbus within a network of industrial PLCs. Despite this, we expanded an existing library, as detailed in the Implementation chapter, thus demonstrating the feasibility of the attack in a similar environment. However, future work could focus on validating the attack on an actual PLC network.

A similar challenge exists for the CRC Manipulation Attack, where we theoretically proved our analysis based on the work and experiments of Åkerberg and Björkman [30]. Here too, testing these techniques in a real-world environment would provide more reliable evidence of the attack's efficacy.

9.2. Future Research

This thesis establishes several pathways for future research to explore, both in attack and defense techniques.

Attacks

- **UART Error Management and Other Protocols:** We focused on PROFIbus and PROFIsafe as our stack on top of UART and RS-485. However, numerous other UART-based protocols might possess vulnerabilities exploitable from generating errors at the data-link layer. These protocols could be found still in industrial environments (e.g., Modbus) or other fields like Military Defense or the Internet of Things.

- **Beyond the Data-Link Level:** As mentioned in the Attacks chapter, an attacker might achieve similar results to our developed attacks, particularly in PROFIbus, potentially without having control at the data-link level, thus having less stringent requirements. Future research could build upon this work to analyze such scenarios.
- **Improve the CRC Management Algorithm:** Our analysis demonstrated the weakness of CRC generation in PROFIsafe, even in its latest version, and presented faster hacking techniques than previously proposed. However, these algorithms may not be optimal. Future research could develop algorithms to find the secret variables even more quickly, thus increasing the threat in critical environments.

Countermeasures

- **Countermeasures Development:** This work introduced three defense mechanisms, one for each level of the stack, that can be implemented to enhance protocol security and prevent the developed attacks. While standard curators should ideally implement and test these countermeasures internally, research could further explore and optimize these techniques.
- **IDS Implementation:** We defined the behavior of an Intrusion Detection System to detect the Reparameterization Attack against PROFIbus. Future work could implement and test this system to prove its effectiveness.

10 | Conclusions

The primary aim of this thesis was to investigate and exploit vulnerabilities related to error management within the widely adopted industrial communication protocols UART, PROFIbus, and PROFIsafe. The research led to the development of novel attack strategies that can compromise the integrity and availability of industrial systems. The focus on the data link layer, where the UART protocol resides, derived from its foundational role in communication within many ICSs. The thesis further developed practical methods, implementing the attack strategies, to exploit the vulnerabilities and assess their impact on the application and system levels.

The research journey encompassed an extensive study of protocol standards, vendor manuals, implementations, and industrial communication and safety standards. The complexities of PROFIbus and PROFIsafe implementations, often proprietary and closed-source, necessitated the extension of the open-source PyProfibus library to incorporate a PROFIbus Device stack and implement essential PROFIsafe features. This enabled the testing of exploits in a controlled, simulated environment that closely replicated a real-world fieldbus network within an ICS.

The experimental validation, particularly focusing on the Reparameterization Attack against PROFIbus, showcased how vulnerabilities in error management mechanisms can be exploited to disrupt communication, manipulate data, and even gain unauthorized control of industrial processes. The findings underscored the feasibility and potential impact of such attacks, highlighting the urgent need for enhanced security measures in industrial communication protocols.

The thesis makes several significant contributions to the field of ICS security. The introduction of the Reparameterization Attack, exploiting the reparameterization vulnerability in the PROFIbus protocol, and the enhancement of an existing attack on PROFIsafe through efficient CRC2 checksum manipulation, demonstrate novel strategies that can compromise critical systems. The proposed countermeasures, including arbitration mechanisms at the UART/RS-485 level, the use of static parameters in PROFIbus, and frequent random key exchanges in PROFIsafe, offer concrete solutions to address the identified vul-

nerabilities.

The limitations of this research, primarily related to testing constraints due to the proprietary nature of many implementations, open the way for future work. Validating the attacks in fully third-party environments and exploring the potential for similar exploits in other UART-based protocols represent promising avenues for further investigation. The development and implementation of the proposed Intrusion Detection System, along with continued research into enhancing CRC management algorithms and optimizing countermeasures, are crucial steps towards fortifying the security of industrial communication systems.

In conclusion, the increasing reliance on Industrial Control Systems (ICSs) in critical infrastructure and the evolving landscape of modern warfare have made cyberattacks against these systems increasingly attractive. The potential for disruption, economic damage, and even threats to human life makes ICSs a prime target for malicious actors, ranging from nation-states to cybercriminals. The attacks presented in this thesis, while relatively simple in concept, highlight the alarming ease with which even attackers with limited resources can exploit vulnerabilities in widely used industrial communication protocols. The lack of robust security measures in legacy systems, coupled with the expanding attack surface due to increased connectivity, paints a concerning picture for the future. The potential consequences of successful attacks on ICSs are far-reaching, impacting not only industrial production and economic stability but also essential services and, most importantly, human safety. The path forward necessitates a proactive and multi-layered approach to ICS security, encompassing both technological advancements and robust operational practices. The development and implementation of effective countermeasures, coupled with continuous research and vigilance, are paramount in safeguarding critical infrastructure and ensuring a secure and resilient industrial landscape.

Bibliography

[1] Industrial communication networks - fieldbus specifications - part 5-3: Application layer service definition - type 3 elements, October 2014.

[2] Industrial communication networks - fieldbus specifications - part 6-3: Application layer protocol specification - type 3 elements, August 2019.

[3] Industrial communication networks - profiles - part 3-3: Functional safety fieldbuses - additional specifications for CPF 3, June 2021.

[4] S. AG. *Fail-safe Module F-CM AS-i Safety ST (3RK7136-6SC00-0BC1)*, 03/2017 edition, 3 2017.

[5] S. AG. *S7-1200 Functional Safety Manual.* NÜRNBERG, GERMANY, 09/2023 edition, 9 2023.

[6] S. AG. *SIMATIC Safety - Configuring and Programming.* NÜRNBERG, GERMANY, 11/2023 edition, 11 2023.

[7] A. Alfonsi. Pyprofisafe. https://github.com/alealfonsi/pyprofisafe.

[8] Buesch. Pyprofibus. https://bues.ch/cms/automation/profibus.

[9] I. E. Commission. Industrial communication networks - profiles - part 3-3: Functional safety profiles for profibus and profinet s2, 2021.

[10] G. Y. Dayanıklı, A. Z. Mohammad, R. Gerdes, and M. Mina. Wireless manipulation of serial communication. In *Proceedings of the 2022 ACM Asia Conference on Computer and Communications Security*, pages 222–236, 2022.

[11] D. Fauri, B. de Wijs, J. den Hartog, E. Costante, E. Zambon, and S. Etalle. Encryption in ICS networks: A blessing or a curse? In *2017 IEEE International Conference on Smart Grid Communications (SmartGridComm)*, pages 289–294. IEEE, 2017.

[12] Hms. Industrial network shares according to hms, 2015. `https://www.automationinside.com/article/industrial-network-shares-according-to-hms`.

[13] Hms. Annual analysis reveals steady growth in industrial network market, 2024. https://www.hms-networks.com/news/news-details/17-06-2024-annual-analysis-reveals-steady-growth-in-industrial-network-market.

[14] E. Hotellier, F. Sicard, J. Francq, and S. Mocanu. Standard specification-based intrusion detection for hierarchical industrial control systems. *Information Sciences*, 659:120102, 2024.

[15] S. Huang, C.-J. Zhou, S.-H. Yang, and Y.-Q. Qin. Cyber-physical system security for networked industrial processes. *International Journal of Automation and Computing*, 12(6):567–578, 2015.

[16] M. Krotofil and D. Gollmann. Industrial control systems security: What is happening? In *2013 4th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)*. IEEE, 2013.

[17] Z. Li, Q. Wei, R. Ma, Y. Geng, Y. Yang, and Z. Lv. Dpguard: A lightweight attack detection method for an industrial bus network. *Electronics*, 12(5):1121, 2023.

[18] S. Mishra, A. Ray, M. Singh, S. Venkatesan, and A. S. Anand. Automated hardware auditing testbed for uart and spi based iot devices. In *2023 10th International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*, pages 75–82, 2023.

[19] Y. Mo and B. Sinopoli. Secure control against replay attacks. In *2009 47th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 911–918. IEEE, 2009. doi: 10.1109/ALLERTON.2009.5394956.

[20] Y. Mo, S. Weerakkody, and B. Sinopoli. Physical authentication of control systems: designing watermarked control inputs to detect counterfeit sensor outputs. *IEEE Control Systems Magazine*, 35(1):93–109, 2015. doi: 10.1109/MCS.2014.2364463.

[21] T. Morris and W. Gao. Industrial control system cyber attacks. In *2013 5th International Conference on Critical Infrastructure (CRIS)*, pages 22–29, 2013.

[22] H. Ochiai, M. D. Hossain, P. Chirupphapa, Y. Kadobayashi, and H. Esaki. Modbus/rs-485 attack detection on communication signals with machine learning. *IEEE Communications Magazine*, 61(6):43–49, 2023.

[23] C. Reichenbach, J. Endresen, M. M. R. Chowdhury, and J. Rossebø. A pragmatic approach on combined safety and security risk analysis. *IEEE Transactions on Electromagnetic Compatibility*, pages 22–27, 2012.

[24] A. Rezai, P. Keshavarzi, and Z. Moravej. Key management issue in scada networks: A review. *Engineering Science and Technology, an International Journal*, 20:354–363, 2017.

[25] J. Rubio-Hernán, L. De Cicco, and J. García-Alfaro. Revisiting a watermark-based detection scheme to handle cyber-physical attacks. In *2016 11th International Conference on Availability, Reliability and Security (ARES)*. IEEE, 2016.

[26] E. Tomur, U. Gulen, E. U. Soykan, M. A. Ersoy, F. Karakoc, L. Karacay, and P. Comak. Sok: Investigation of security and functional safety in industrial iot. In *2021 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 2021.

[27] L. Wang. Access control attacks on plc vulnerabilities, 2018. `https://www.scirp.org/journal/paperinformation?paperid=88844`.

[28] R. Watson and C.-W. Ten. On the security of profibus. *International Journal of Critical Infrastructure Protection*, 18:1–13, 2017.

[29] Y. Zhou, D. Chai, M. Liu, F. Lin, W. Shang, and L. Wang. Research on the security mechanism for interconnection between profibus and internet. In *2014 11th World Congress on Intelligent Control and Automation*. IEEE, 2014.

[30] J. Åkerberg and M. Björkman. Exploring network security in profisafe. In *2009 33rd Annual IEEE International Computer Software and Applications Conference (COMPSAC)*, volume 2, pages 406–412, 2009.

List of Figures

2.1 Industrial Control System overview 7
2.2 PROFIbus frame format 9
2.3 PROFIsafe frame format 12
2.4 UART frame format 14
2.5 RS-485 standard 15

3.1 SCADA attacks classification (Morris) 22

4.1 Typical network topology of a ICS with PROFIbus 30

5.1 Attack scenario 39
5.2 Reparameterization Attack Flow Diagram 41
5.3 Fields that cannot have value '0' 43
5.4 Parameterization Telegram DU Byte 1 44
5.5 CRC2 Calculation 48
5.6 Akerberg's Algorithm 49
5.7 New CRC Manipulation in PROFIsafe 50
5.8 New CRC Manipulation in PROFIsafe from version 2.6.1 54

7.1 PyProfisafe state diagram 65
7.2 Hardware network 66

8.1 Attacker test code 68
8.2 Capture: Attacker overwrites the Controller's telegram 69
8.3 Capture: Victim receives the corrupted telegram 70
8.4 Victim's log 70
8.5 Parameterization Telegram Code 71
8.6 The Attacker reparameterize the Victim 71

List of Tables

3.1 Protocols and their associated attacks. 24

4.1 Threats to ICS by Different Actors 32

5.1 New VCN Management 52

6.1 Countermeasures 58

Acknowledgements